

JAVA UNIT - 1

History & Evolution

- Java is related to C++, which is a direct descendant of C. Much of the character of Java is inherited from these two languages. From C, Java derives its syntax. Many of Java's object oriented features were influenced by C++.
- Early computer languages such as **BASIC**, **COBOL** and **FORTRAN** were not designed around structured principles. Instead, they relied upon the GOTO as a primary means of program control. As a result, programs written using these languages tended to produce a mass of tangled jumps and conditional branches that make a program virtually impossible to understand. While languages like **Pascal** are structured, they were not designed for efficiency, and failed to include certain features necessary to make them applicable to a wide range of programs.
- The **creation of C** is considered by many to have marked the beginning of the modern age of computer languages. It was a programmer's language meaning it was designed, implemented and developed by real, working programmers, tested, thought about, and rethought by the people who actually used the language. The result was a language that programmers liked to use. As such, C found wide and rapid acceptance in the programmer community. In short, C is a language designed by and for programmers.

- Throughout the history of programming, the increasing complexity of programs has driven the need for better ways to manage that complexity. although C is one of the world's great programming languages, there is a limit to its ability to handle complexity. Once the size of a program exceeds a certain point, it becomes so complex that it is difficult to grasp as a totality.
- **C++ extends C** by adding object-oriented features. Because C++ is built on the foundation of C, it includes all of C's features, attributes and benefits.
- The trouble with C and C++ (and most other languages) is that they are designed to be compiled for a specific target. Although it is possible to compile a C++ program for just about any type of CPU, to do so requires a full C++ compiler targeted for that CPU, but these compilers are expensive and time-consuming to create. **Java** enhanced and refined the object-oriented paradigm used by C++, added integrated support for multithreading (which allows you to write programs that do many things simultaneously) and Architecture-Neutral (code longevity and portability i.e. Platform - Independent Language).

The Creation of Java

- Java was developed by James Gosling and his team (Patrick Naughton, Chris Warth, Ed Frank, and Mike Sheridan) at Sun Microsystems, Inc. in 1991.
 - It took 18 months to develop the first working version. This language was initially called “Oak,” but due to trademark issues with “Oak” , it was renamed and released as “Java” in 1995.
 - James Gosling is called “the father of Java”.
 - Primary motivation was the need for a platform-independent(that is, architecture-neutral) language that could be used to create software to be embedded in various consumer electronic devices, such as microwave ovens and remote controls.
 - The problem is that compilers are expensive and time-consuming to create.
 - Solution, a portable, platform-independent language
-
- The second force was, of course, the World Wide Web.
 - The emergence of the World Wide Web, Java was propelled to the forefront of computer language design, because the Web, too, demanded portable programs.
 - The Internet ultimately led to Java’s large-scale success.
 - Java is simply the “Internet version of C++.”
 - Java was to Internet programming what C was to system programming: a revolutionary force that changed the world.

Byte Code

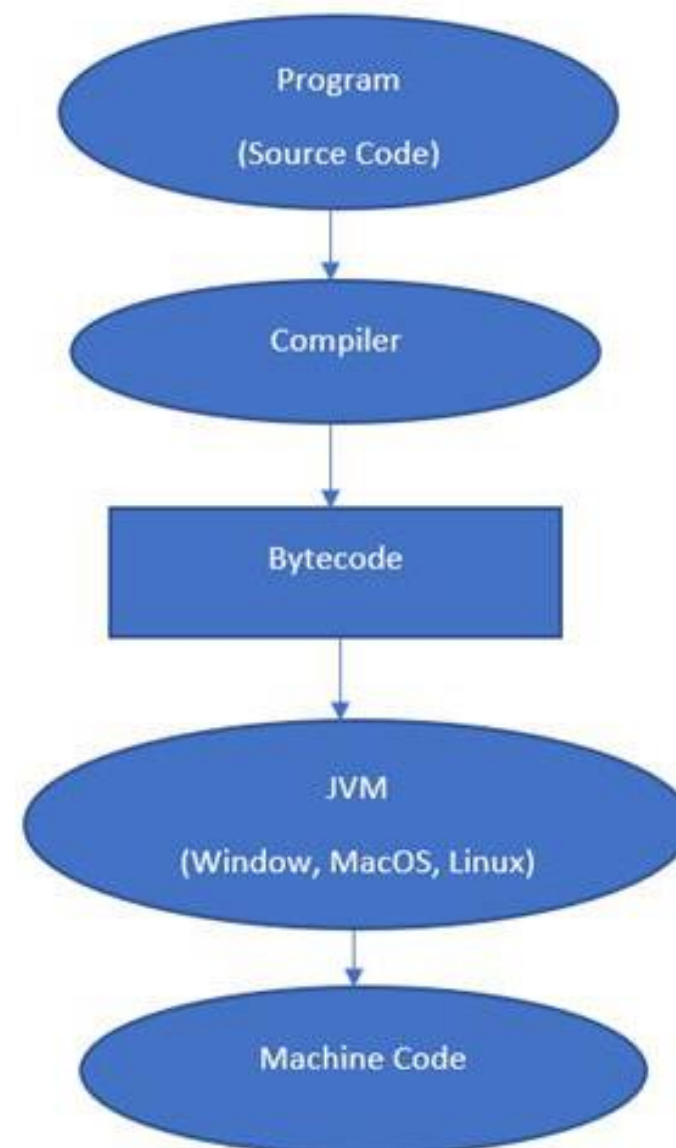
Bytecode is a highly optimized set of instructions designed to be executed by the Java run-time system, which is called Java Virtual Machine (JVM). Bytecode can also be defined as intermediate code generated by compiler after the compilation of source code.

Benefits :

- This intermediate code makes Java a platform-independent language.
- Translating a Java program into bytecode makes it much easier to run a program in a wide variety of environments because only the JVM needs to be implemented for each platform.
- The fact that a Java program is executed by the JVM also helps to make it secure.
- Bytecode has been highly optimized, the use of bytecode enables the JVM to execute programs much faster than you might expect.

Byte Code

Compiler converts the source code or the Java program into the Byte Code(or machine code), and secondly, the Interpreter executes the byte code on the system. The Interpreter can also be called JVM(Java Virtual Machine). The byte code is the common piece between the compiler(which creates it) and Interpreter (which runs it).



Buzzwords

- **Simple**

An experienced C++ programmer, moving to Java will require very little effort because Java inherits C/C++ syntax and many of object-oriented features of C++, most programmers have little trouble learning Java. Java is designed to be easy for beginners and professional programmers to learn and use effectively. Java has removed many complicated and rarely-used features, such as explicit pointers and operator overloading.

- **Object-Oriented**

Everything in Java revolves around objects and classes. Java allows you to model real-world entities (like a car or a bank account) as objects in your program, making it easier to manage and build complex applications.

Key Object-Oriented Programming (OOP) concepts include:

1. **Object:** An instance of a class.
2. **Class:** A blueprint for creating objects.
3. **Inheritance:** Allows one class to inherit the properties of another.
4. **Polymorphism:** The ability of objects to take on multiple forms.
5. **Abstraction:** Hides the complex details and shows only the essentials.
6. **Encapsulation:** Keeps the data safe by restricting access to it.

Buzzwords

- **Robust**
 - Java provides many features that make programs execute reliably in a variety of environments. Java is a strictly typed language that checks code at compile time and runtime. In C/C++, the programmer will often manually allocate and free all dynamic memory. Java virtually eliminates these problems by managing memory allocation and deallocation for you. Java handles memory management with garbage collection and captures serious errors through exception handling.
 - Program must execute reliably in a variety of systems.
 - Java is a strictly typed language, it checks your code at compile time. However, it also checks your code at run time.
 - Reliability
 - a. early (compile time) checking
 - b. dynamic (runtime) checking
 - c. eliminating situations that are error prone.
 - Example: Two of the main reasons for program failure: memory management mistakes and mishandled exceptional conditions (that is, run time errors).

Buzzwords

- **Multithreaded**

Multithreaded programs handle multiple tasks simultaneously, which is helpful in creating interactive, networked programs. Java was designed to meet the real-world requirement of creating interactive, networked programs. To accomplish this, Java supports multithreaded programming which allows you to write programs that do many things simultaneously. Java's easy-to-use approach to multithreading allows you to think about the specific behavior of your program, not the multitasking subsystem.

- Java supports multithreaded programming, which allows you to write programs that do many things simultaneously.

- A thread is a part of the program that can operate independently of its other parts

- Multi threaded programs can do multiple things at once
example: download a file from the web while still looking at other web pages

Buzzwords

- **Architecture-neutral**

A central issue for the Java designers was that of code longevity and portability. Changes and upgrades in operating systems, processors, and system resources do not force any changes in Java programs. The Java designers made several hard decisions in the Java language and the Java Virtual Machine in an attempt to alter this situation. Their goal was “write once; run anywhere, any time, forever.”

- Goal was WORA - write once; run anywhere, any time, forever.
- One of the main problems facing programmers is that no guarantee exists that if you write a program today, it will run tomorrow even on the same machine.
- Operating system upgrades, processor upgrades, and changes in core system resources can all combine to make a program malfunction.

Buzzwords

- **Compiled & Interpreted**

Java combines both compiled and interpreted approaches, making it a two-stage system.

1. **Compiled:** Java compiles programs into an intermediate representation called Java Bytecode.
2. **Interpreted:** Bytecode is then interpreted, generating machine code that can be directly executed by the machine that provides a JVM.

- **High performance**

Java performance is high because of the use of bytecode. The bytecode can be easily translated into native machine code.

- Java enables creation of cross platform programs by compiling into an intermediate representation called Java bytecode . This code can be executed on any system that implements Java Virtual Machine.

- Most previous attempts at cross platform solutions have done so at expense of performance.

- Java run time systems that provide this feature lose none of benefits of platform independent code.

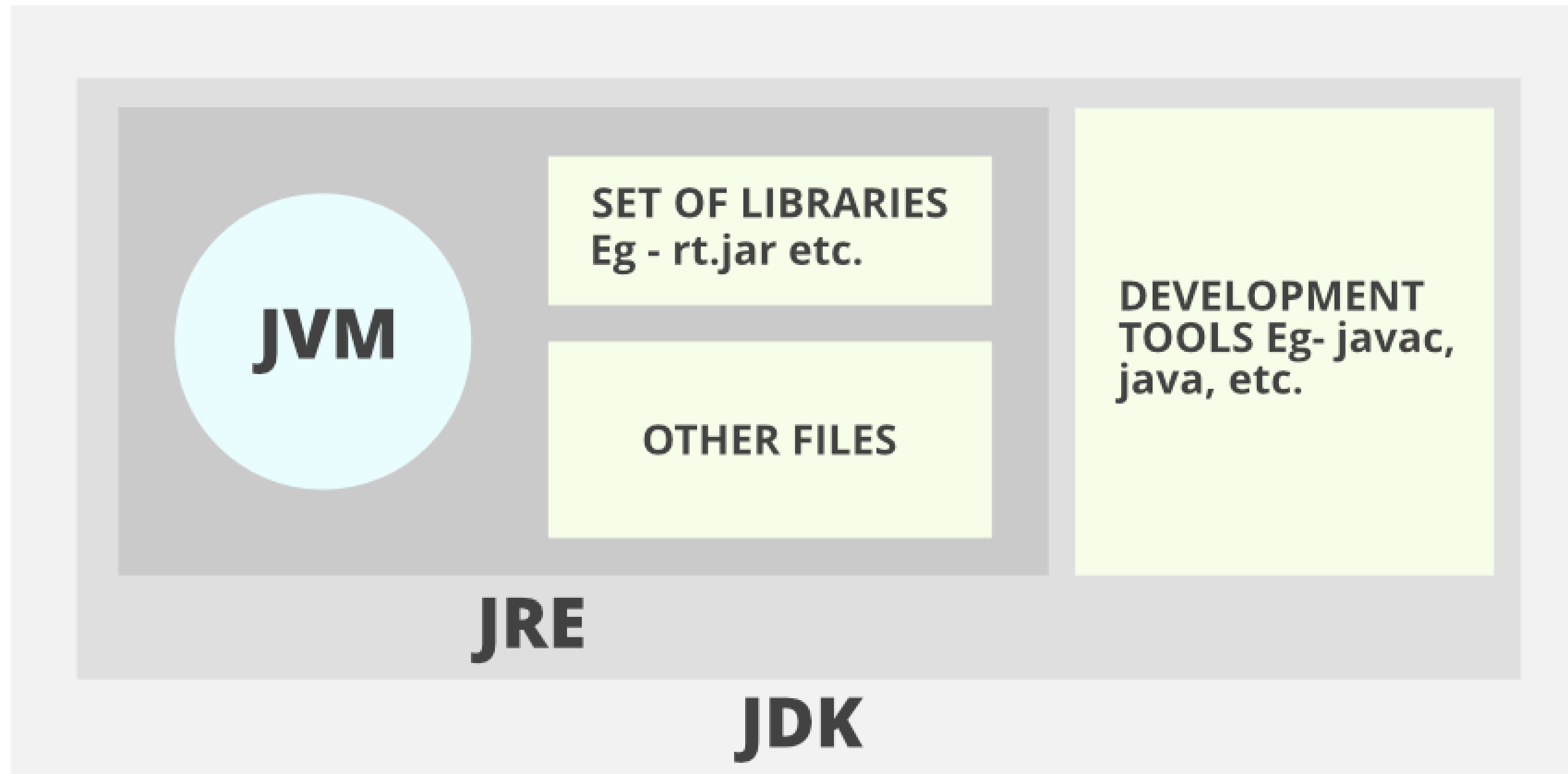
Buzzwords

- **Distributed**
Java is designed to create distributed applications on networks. Java applications can access remote objects on the Internet as easily as they can do in the local system. Java enables multiple programmers at multiple remote locations to collaborate and work together on a single project.
- **Dynamic**
Java can link in new class libraries, methods, and objects dynamically. Java programs carry substantial amounts of run-time type information, enabling dynamic linking in a safe and expedient manner.

Buzzwords

- **Secure**
Java does not use pointers, which helps prevent unauthorized memory access. The JVM verifies Java bytecode before execution, ensuring that it adheres to Java's security constraints. Java applications run in a restricted environment (sandbox) that limits their access to system resources and user data, enhancing security.
- **Portable**
Java provides a way to download programs dynamically to various types of platforms connected to the Internet. Java is portable because of the JVM, which provides a consistent runtime environment across different platforms.

Java Environment



Java Environment

- **JDK: Java Development Kit** - The Java Development Kit (JDK) is a cross-platformed software development environment that offers a collection of tools and libraries necessary for developing Java-based software applications and applets. It is a core package used in Java, along with the JVM (Java Virtual Machine) and the JRE (Java Runtime Environment). JDK contains:
 1. Java Runtime Environment (JRE)
 2. An interpreter/loader (Java)
 3. A compiler (javac)
 4. An archiver (jar) and others.Most Popular JDKs: Oracle JDK, OpenJDK, IBM J9 JDK.
- **JRE: Java Runtime Environment** - Java Runtime Environment (JRE) is an open-access software distribution that has a Java class library, specific tools, and a separate JVM. Java source code is compiled and converted to Java bytecode. If you want to run this bytecode on any platform, you need JRE. The JRE loads classes check memory access and get system resources.

Java Environment

The JRE runtime architecture consists of the following elements as listed:

1. ClassLoader
 2. ByteCode verifier
 3. Interpreter
- **JVM : Java Virtual Machine** - JVM stands for Java Virtual Machine. JVM is used for running Java bytecode. Java applications are called WORA (Write Once Run Anywhere). This means a programmer can develop Java code on one system and can expect it to run on any other Java-enabled system without any adjustment. This is all possible because of JVM.

Installation of Eclipse

For JDK:

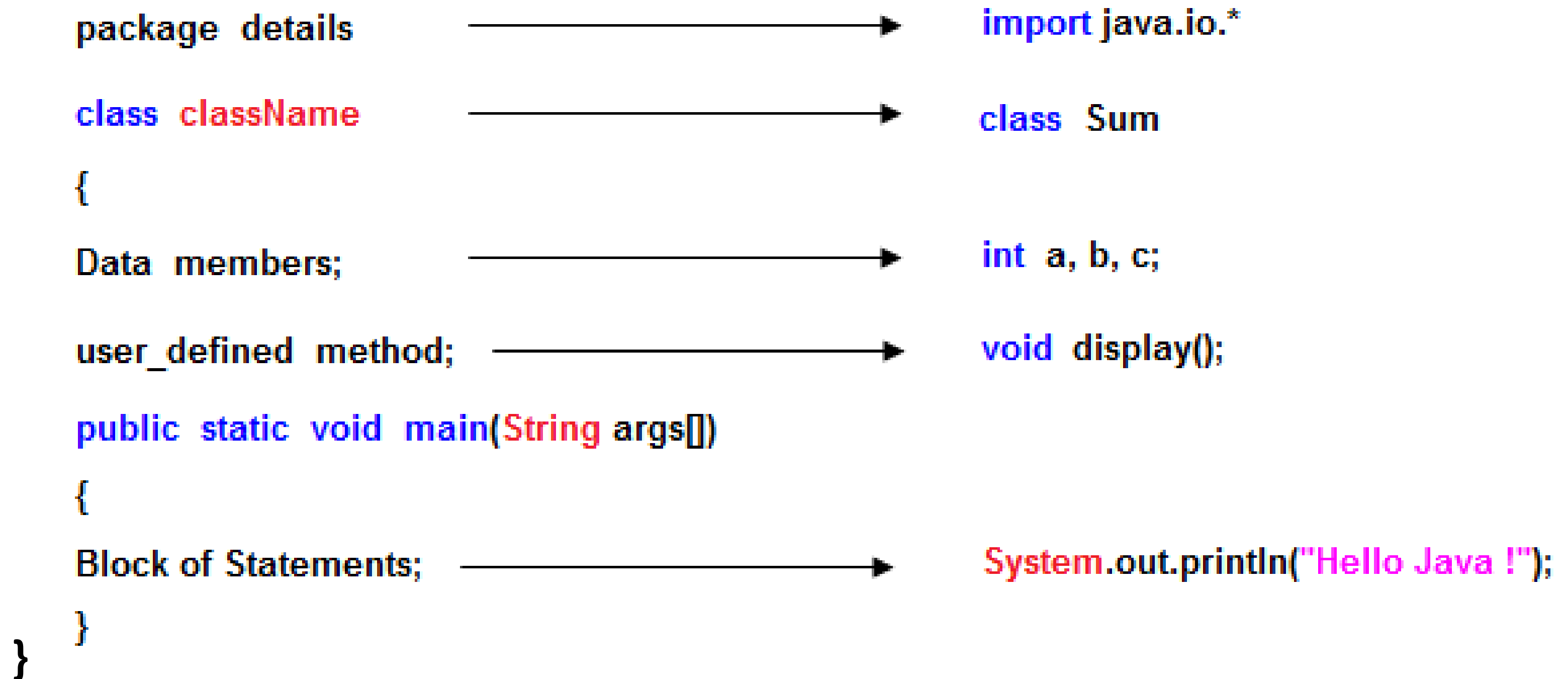
- Visit Oracle JDK Downloads.
- Under Windows , download 64 Installer.
- Once downloaded, double click ~ next ~ next ~ succesfully installed.

For Eclipse IDE:

- Visit eclipse.org , click download.
- Under Install your favorite desktop IDE packages, click download.
- In windows search box, search environment variables.
- Select environment variables
- Under system variables ~ new ~
 Name -JAVA_HOME
 Value-path to jdk folder
- Under system variables ~ find path ~ edit ~ new ~ %JAVA_HOME%\bin
- Check ~ command prompt ~ type ~
 java -version
 javac- version
- Open eclipse now.
- Launch Eclipse
- Create package and class.

Structure of Java Program

Documentation Section (Optional)



Fundamental rules and conventions in Java

Core Rules:

- **Case Sensitivity** : Java is case-sensitive. myVariable, MyVariable, and MYVARIABLE are treated as distinct identifiers.
- **Semicolons** : Every statement in Java must end with a semicolon (;).
- **Classes and Objects** : Java is an object-oriented language. All code resides within classes, and programs often interact by creating and manipulating objects (instances of classes).
- **Main Method** : Every executable Java application must have a public static void main(String[] args) method. This is the entry point where the program begins execution.
- **File Naming** : A public class within a Java source file must have the same name as the file, including the .java extension (e.g., MyClass.java contains public class MyClass).
- **Curly Braces** : Curly braces ({}) are used to define code blocks, such as class bodies, method bodies, and control flow statements (e.g., if, for, while).

Structure of Java Program

Comments in Java

There are three types of comments in Java:

- **Single-line comments:** These comments begin with `//` and extend to the end of the line.
- **Multi-line comments (or block comments):** These comments start with `/*` and end with `*/`. They can span multiple lines and are used for longer explanations, commenting out blocks of code, or providing detailed descriptions of methods or classes.
- **Documentation comments (or Javadoc comments):** These special comments start with `/**` and end with `*/`. They are used to generate API documentation (comprehensive guide or manual to explain use and interaction with an API) using the Javadoc tool. They typically precede class, method, or field declarations and can include specific Javadoc tags (e.g., `@param`, `@return`) to describe parameters, return values, and other aspects of the code.

Multi-line are for general-purpose commenting of multiple lines of code, used by developers and ignored during compilation.

While documentation comments are multi-line comments for external API documentation for the project and are processed by the javadoc to generate formal documentation.


```
package org.argouml.model;

import java.beans.PropertyChangeListener;

/**
 * Abstract class that implements the convenience methods of the
 * {@link ModelEventPump} interface.
 *
 * @author Linus Tolke
 */
public abstract class AbstractModelEventPump implements ModelEventPump {
    /**
     * @see org.argouml.model.ModelEventPump#addModelEventListener(
     *      java.beans.PropertyChangeListener, java.lang.Object,
     *      java.lang.String)
     */
    public void addModelEventListener(PropertyChangeListener listener, Object modelement, String eventName) {
        addModelEventListener(listener, modelement, new String[] {eventName});
    }
}
```


Structure of Java Program

public static void main(String args[])

- The public keyword is an *access specifier, which allows programmer to control* visibility of class members.
- When a class member is preceded by **public**, then that member may be accessed by code outside the class in which it is declared.
- The keyword **static** allows main() to be called without having to instantiate a particular instance of the class.
- This is necessary since **main()** is called by the Java Virtual Machine before any objects are made.
- The keyword **void** simply tells the compiler that main() does not return a value.
- In main(), there is only one parameter, String args[] declares a parameter named args, which is an array of instances of the class String.

Using Blocks of Code

- Java allows two or more statements to be grouped into *blocks of code, also called code blocks*.
- This is done by enclosing the statements between opening and closing curly braces.
- Once a block of code has been created, it becomes a logical unit that can be used any place that a single statement can.

```
if(x < y) { // begin a block
```

```
x = y;
```

```
y = 0;
```

```
} // end of block
```


Structure of Java Program

Lexical Issues

- Java programs are a collection of whitespace, identifiers, literals, comments, operators, separators, and keywords.

- **Whitespace**

- Java is a free-form language. This means that you do not need to follow any special indentation rules.

- In Java, whitespace is a space, tab, or newline.

- **Identifiers**

- Identifiers are used for class names, method names, and variable names.

- An identifier may be any descriptive sequence of uppercase and lowercase letters, numbers, or the underscore and dollar-sign characters.

- They must not begin with a number.

- Java is case-sensitive.

- **Literals**

- A constant value in Java is created by using a *literal representation of it*.

- **Comments**

- Three types of comments defined by Java.

Structure of Java Program

•Separators

- In Java, there are a few characters that are used as separators.
- The most commonly used separator in Java is the semicolon.

•The Java Keywords

- There are 50 keywords currently defined in the Java language
- These keywords cannot be used as names for a variable, class, or method.
- In addition to the keywords, Java reserves the following: **true**, **false**, and **null**. These are values defined by Java.

Symbol	Name	Purpose
()	Parentheses	Used to contain lists of parameters in method definition and invocation. Also used for defining precedence in expressions, containing expressions in control statements, and surrounding cast types.
{ }	Braces	Used to contain the values of automatically initialized arrays. Also used to define a block of code, for classes, methods, and local scopes.
[]	Brackets	Used to declare array types. Also used when dereferencing array values.
;	Semicolon	Terminates statements.
,	Comma	Separates consecutive identifiers in a variable declaration. Also used to chain statements together inside a for statement.
.	Period	Used to separate package names from subpackages and classes. Also used to separate a variable or method from a reference variable.

abstract	continue	for	new	switch
assert	default	goto	package	synchronized
boolean	do	if	private	this
break	double	implements	protected	throw
byte	else	import	public	throws
case	enum	instanceof	return	transient
catch	extends	int	short	try
char	final	interface	static	void
class	finally	long	strictfp	volatile
const	float	native	super	while

Object-Oriented Programming

- **Two Paradigms**

- All computer programs consist of two elements: code and data.

- Furthermore, a program can be conceptually organized around its code or around its data. That is, some programs are written around “what is happening” and others are written around “who is being affected.”

- These are the two paradigms that govern how a program is constructed.

- *process-oriented model*

- The process-oriented model can be thought of as *code acting on data*.

- C employ this model.

- *object-oriented programming*

- Object-oriented programming organizes a program around its data (that is, objects) and a set of well-defined interfaces to that data.

- An object-oriented program can be characterized as *data controlling access to code*.

Abstraction

- An essential element of object-oriented programming is *abstraction*.
- *Humans manage* complexity through abstraction.
- Example, people do not think of a car as a set of tens of thousands of individual parts. Instead, they are free to utilize the object as a whole.
- A powerful way to manage abstraction is through the use of hierarchical classifications.
- This allows you to layer the semantics of complex systems, breaking them into more manageable pieces.
- Hierarchical abstractions of complex systems can also be applied to computer programs.
- The data from a traditional process-oriented program can be transformed by abstraction into its component objects.
- A sequence of process steps can become a collection of messages between these objects. Thus, each of these objects describes its own unique behavior.
- You can treat these objects as concrete entities that respond to messages telling them to *do something*.
- This is the essence of object-oriented programming.
- Object-oriented programming is a powerful and natural paradigm for creating programs that survive the inevitable changes accompanying the life cycle of any major software project, including conception, growth, and aging.

Encapsulation

- *Encapsulation is the mechanism that binds together code and the data it manipulates, and keeps both safe from outside interference and misuse.*
- One way to think about encapsulation is as a protective wrapper that prevents the code and data from being arbitrarily accessed by other code defined outside the wrapper.
- To relate this to real world, You can't affect transmission by using turn signal or windshield wipers, for example.
- A *class defines* the structure and behavior (data and code) that will be shared by a set of objects.
- Each object of a given class contains the structure and behavior defined by the class, as if it were stamped out by a mold in the shape of the class.
- For this reason, objects are sometimes referred to as *instances of a class*.
- *Thus, a class is a logical construct; an object has physical reality.*

Encapsulation

- When you create a class, you will specify the code and data that constitute that class. Collectively, these elements are called *members of the class*.
- *Specifically, the data defined by the class are referred to as member variables or instance variables. The code that operates on that data is referred to as member methods or just methods.*
- Since the purpose of a class is to encapsulate complexity, there are mechanisms for hiding the complexity of the implementation inside the class.
- Each method or variable in a class may be marked private or public.

Inheritance

- *Inheritance is the process by which one object acquires the properties of another object. This* is important because it supports the concept of hierarchical classification.
- Inheritance, an object need only define those qualities that make it unique within its class. It can inherit its general attributes from its parent.
- Thus, it is the inheritance mechanism that makes it possible for one object to be a specific instance of a more general case.
- A deeply inherited subclass inherits all of the attributes from each of its ancestors in the *class hierarchy*.
- Inheritance interacts with encapsulation as well. If a given class encapsulates some attributes, then any subclass will have the same attributes *plus any that it adds as part of its* specialization.
- This is a key concept that lets object-oriented programs grow in complexity linearly rather than geometrically.
- A new subclass inherits all of the attributes of all of its ancestors. It does not have unpredictable interactions with the majority of the rest of the code in the system.

Polymorphism

- *Polymorphism (from Greek, meaning “many forms”) is a feature that allows one interface to be used for a general class of actions.*
- Ex:-Consider a stack (which is a last-in, first-out list).
- The concept of polymorphism is often expressed by the phrase “one interface, multiple methods.”
- This means that it is possible to design a generic interface to a group of related activities.
- This helps reduce complexity by allowing the same interface to be used to specify a *general class of action*.
- *It is the compiler’s job to select the specific action* (that is, method) as it applies to each situation. You, the programmer, do not need to make this selection manually. You need only remember and utilize the general interface.
- Example: a dog’s sense of smell is polymorphic.

Java Program

1

Program named WelcomeMessage to print Welcome to Java Programming!

2

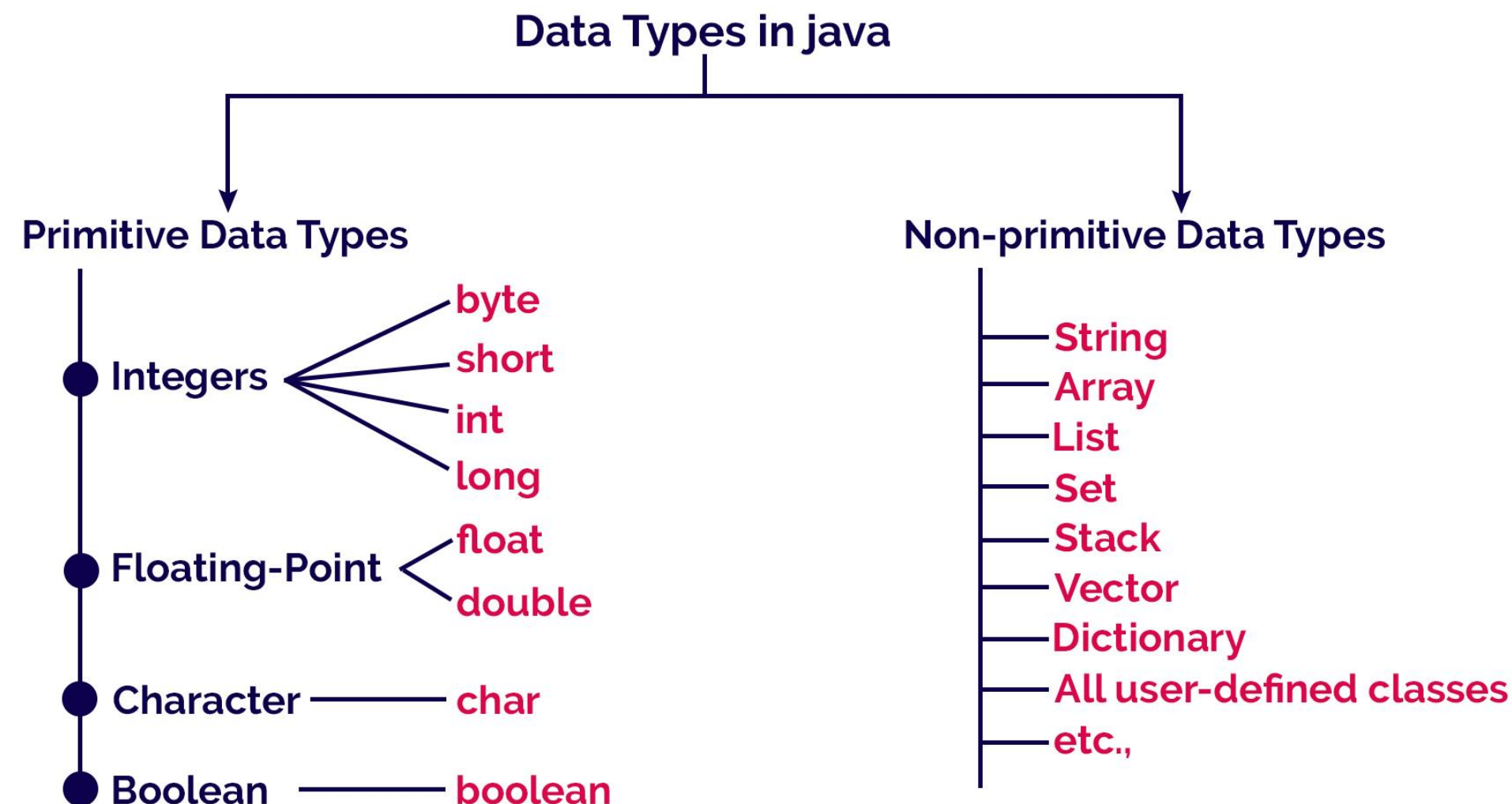
Program named StudentInfo that stores and displays the following details :

- Name
- Gender
- Class & Section
- URN
- CRN
- Age
- College Name & Address

Data types in Java

Java is statically typed programming language which means variable types are known or checked at the compile time, before the program is executed.

- In Java, compiler knows exactly what types each variable holds and enforces correct usage during compilation. For example "int x = "GfG";" gives compiler error in Java as we are trying to store string in an integer type.
- Data types in Java are of different sizes and values that can be stored in a variable that is made as per convenience and circumstances to handle different scenarios or data requirements.



Data types in Java

Primitive Data Types :

The primitive data types are built-in data types and they specify the type of value stored in a variable and the memory size. The primitive data types do not have any additional methods. Eg - byte, short, int, long, float, double, char, and boolean.

Non-primitive Data Types :

In java, non-primitive data types are the reference data types or user-created data types. All non-primitive data types are implemented using object concepts. The non-primitive data types may use additional methods to perform certain operations. The default value of non-primitive data type variable is null. Eg - String, Array, List, Queue, Stack, Class, Interface, etc. Non-primitive data types are called reference types because they refer to objects.

Primitive Data Type	Non-primitive Data Type
These are built-in data types	These are created by the users
Does not support additional methods	Support additional methods
Always has a value	It can be null
Starts with lower-case letter	Starts with upper-case letter
Size depends on the data type	Same size for all

Primitive Data types in Java

Data Type	Size	Example	Description
byte	1 byte / 8 bits	30, -97	Stores whole number from -128 to 127
short	2 bytes / 16 bits	5000, -13,848	Stores whole number from -32,768 to 32,767
int	4 bytes / 32 bits	100000, 0, -2	Stores whole number from -2,147,483,648 to 2,147,483,647
long	8 bytes / 64 bits	78000000000L, 0L, -2L	Stores whole numbers from -9,233,372,036,854,755,808 to 9,233,372,036,854,755,807
float	4 bytes / 32 bits	3.14f, -1.23e-10f	Stores fractional numbers. Sufficient for storing 7 decimal digits
double	8 bytes / 64 bits	3.1415d, 1.23e100d	Stores fractional numbers. Sufficient for storing 15 decimal digits
boolean	1 bit	true, false	Stores true or false values
char	2 bytes / 16 bits	J, Q	Stores Single character / letter

Non - Primitive Data types in Java

These data types do not directly hold the value but instead hold a reference (memory address) to an object.

- String:** Represents a sequence of characters (text). Enclosed in double quotes.

```
String name = "Alice";
```

- Arrays:** A collection of elements of the same data type.

```
int[] numbers = {1, 2, 3, 4, 5};
```

- Classes & Objects:** User-defined blueprints for creating objects. String is an example of a built-in class.

```
// Assuming a 'Person' class exists
```

```
Person person1 = new Person("Bob", 25);
```

- Interfaces:** Define a contract for classes, specifying methods that a class must implement.

```
// Assuming a 'Runnable' interface exists
```

```
Runnable task = () -> System.out.println("Running a task.");
```

JAVA Lecture -1.3

August 2025

JAVA VARIABLES

In Java, variables are containers that store data in memory. Understanding variables plays a very important role as it defines how data is stored, accessed, and manipulated.

Key Components of Variables in Java:

A variable in Java has three components, which are listed below:

- **Data Type:** Defines the kind of data stored (e.g., int, String, float).
- **Variable Name:** A unique identifier following Java naming rules.
- **Value:** The actual data assigned to the variable.

Rules for Naming Variables : When naming variables in Java, there are some rules and conventions to follow :

- Variable names must start with a letter, dollar sign (\$), or underscore (_).
- Variable names can contain letters, digits, dollar signs, or underscores, but cannot start with a digit.
- Java is case-sensitive, so age and Age would be treated as two different variables.
- Avoid using reserved keywords (like int, class, public, etc.) as variable names.

JAVA VARIABLES

1). Variable Declaration :

Declaration involves specifying the data type and name of a variable. This reserves a named memory location to store a value of the specified type. No initial value is assigned during declaration alone.

```
int age;
```

```
String name;
```

2). Variable Initialization :

Initialization is the process of assigning an initial value to a declared variable. This can be done in two main ways:

- Explicit Initialization (at declaration): The most common method, where a value is assigned to the variable at the time of its declaration.

```
int score = 100;
```

```
String message = "Hello, Java!";
```

- Implicit Initialization (after declaration): A value can be assigned to a variable after it has been declared, using the assignment operator (=).

```
int count;
```

```
count = 50;
```

JAVA VARIABLES

In Java, initialization refers to the process of assigning an initial value to a variable. This can occur in two primary ways: static initialization and dynamic initialization.

- **Static Initialization** : Static initialization occurs when a variable's value is known and assigned at compile time. This means the value is fixed and does not change during program execution.

Characteristics:

- ☐ Compile-Time Assignment: The value is provided as a literal or a constant expression directly in the declaration.
- ☐ Fixed Value: The variable holds the same value throughout the program's lifecycle.
- ☐ Examples:
 - ☐ Primitive type variables: `int x = 10;`, `double pi = 3.14159;`
 - ☐ Static fields in a class: `public static final String APP_NAME = "MyApplication";`

- **Dynamic Initialization** : Dynamic initialization occurs when a variable's value is determined and assigned at runtime. This means the value can be the result of an expression, a method call, or user input, and thus can vary each time the program runs.

Characteristics:

- ☐ Runtime Assignment: The value is determined during the execution of the program.
- ☐ Variable Value: The value can change based on runtime conditions or computations.
- ☐ Examples:
 - ☐ Assigning the result of a calculation: `int sum = a + b;`
 - ☐ Assigning the return value of a method: `double area = calculateArea(radius);`
 - ☐ Assigning user input: `String name = scanner.nextLine();`

JAVA VARIABLES

Types of Java Variables

- **Local Variables :**
 - ☐ Declared inside a method, constructor, or a block of code.
 - ☐ Their scope is limited to the block in which they are declared, meaning they can only be accessed within that specific method, constructor, or block.
 - ☐ They are created when the method, constructor, or block is entered and destroyed when it exits.
 - ☐ Local variables do not have default values and must be explicitly initialized before use.
- **Instance Variables:**
 - ☐ Declared inside a class but outside of any method, constructor or block.
 - ☐ They are associated with an instance (object) of the class. Each object of the class will have its own copy of the instance variables.
 - ☐ They are created when an object of the class is instantiated and destroyed when object is garbage collected.
 - ☐ They are automatically initialized with default values (e.g., 0 for numeric types, false for booleans, null for reference types) if not explicitly initialized.
- **Static (or Class) Variables:**
 - ☐ Declared inside a class, outside of any method, constructor, or block, and marked with the static keyword.
 - ☐ They are associated with class itself, not with individual objects. All instances of class share same static variable.
 - ☐ They are created when the class is loaded into memory and exist for the entire duration of the program.
 - ☐ Static variables can be accessed directly using class name (e.g., ClassName.variableName) without needing to create an object.
 - ☐ They are also automatically initialized with default values if not explicitly initialized.

JAVA LITERALS

In Java, a literal is a fixed value that is directly represented in the source code. These constant values are used to initialize variables or to specify values within expressions and method calls.

In Java, a Literal is value of boolean, numeric, character, or string data. Any constant value that can be assigned to the variable is called a literal.

```
int x = 92;
```

int is datatype, x is the integer variable and literal '92' represents the value of ninety two.

Therefore, a constant value that is assigned to the variable can be called a literal.

Types of Literals in Java

Java supports the following types of literals:

- Integral Literals
- Floating-Point Literals
- Char Literals
- String Literals
- Boolean Literals

JAVA LITERALS

Integral Literals in Java : Represent whole numbers. For Integral data types (byte, short, int, long), we can specify literals in four ways, which are listed below:

1. **Decimal Integer:** Decimal integers use a base ten and digits ranging from 0 to 9. They can have a negative (-) or a positive (+), but non-digit characters or commas aren't allowed between characters. Example: 2022, +42, -68. Decimal is everyday use, it is our natural number systems.
2. **Octal Integer:** Octal integers use a base eight and digits ranging from 0 to 7. Octal integers always begin with a "0." Example: 007, 0295. Appears in Linux.
3. **Binary Integer:** Binary integers uses a base two, consisting of the digits "0" and "1." The prefix "0b" represents the Binary system. Example: 0b11011. At hardware level.
4. **Hexa-Decimal:** Hexa-decimal integers work with a base 16 and use digits from 0 to 9 and the characters of A through F. The characters are case-sensitive and represent a 10 to 15 numerical range. Example: 0xf, 0xe, 0xFF. Binary gets too long and confusing, so hexadecimal is shortcut for binary.

By default, integer literals are of type int. To specify a long literal, append L or l, e.g., 123L. There is no way to specify byte and short literals explicitly but indirectly we can specify. Whenever we are assigning integral literal to the byte variable and if the value is within the range of byte then the compiler treats it automatically as byte literals.

JAVA LITERALS

Floating-Point Literal in Java

Floating-point literals are expressed as exponential notations or as decimal fractions. They can represent either a positive or negative value, but if it's not specified, the value defaults to positive. Floating-point literals come in these formats:

1. **Floating Point** : Floating format single precision (4 bytes) end with an “f” or “F.” Example: 4f. Floating format double precision (8 bytes) end with a “d” or “D.” Example: 3.14d.
2. **Hexa-Decimal Floating Point** : Also supported in Java, but they are rarely used. Structure of a Hexadecimal Floating-Point Literal:
 - Ox or OX: A prefix indicating a hexadecimal number.
 - hex-digit-sequence: A sequence of hexadecimal digits (0-9 and a-f or A-F) representing the whole number part.
 - . : An optional decimal point.
 - hex-digit-sequence: Another sequence of hexadecimal digits representing the fractional part, if present.
 - p or P: A separator indicating the exponent part.
 - decimal-integer: An optionally signed decimal integer representing the power of two by which the significand is multiplied.

For example $0x1.8p-12$ represents $(1 + 8/16) \times 2^{-12} = 3/8192$, and $0x1.cp12$ represents $(1 + 12/16) \times 2^{12} = 7168$.

By default, every floating-point literal is of double type, and hence we can't assign directly to the float variable without suffix f and F. But we can specify floating-point literal as float type by suffixed with f or F. We can specify explicitly floating-point literal as double type by suffixed with d or D.

JAVA LITERALS

Char Literals

For char data types, we can specify literals in four ways which are listed below:

1. Single quote: A character literal can be any single character (letter, digit, or symbol) enclosed in single quotes.

Eg: char letter = 'A';
char digit = '5';
char symbol = '\$';

2. Unicode Representation: Java is Unicode language meaning it supports all scripts. You can specify a character literal using its Unicode value in the format `\uXXXX`, where XXXX is a four-digit hexadecimal number representing the Unicode code point.

Eg: char omega = '\u03A9'; // Represents the Greek capital letter Omega (Ω)

3. Escape Sequence: Java uses escape sequences to represent special characters that are difficult or impossible to type directly, or characters that have special meaning within the code. These sequences begin with a backslash (`\`). Used for formatting output.

Eg : char newline = '\n'; Represents a newline character
char tab = '\t'; Represents a tab character
char backslash = '\\'; Represents a backslash character

JAVA LITERALS

String Literals

String literals are sequences of characters enclosed between double quote (") marks. These characters can be alphanumeric, special characters, blank spaces, etc.

Examples: "John", "2468", "\n", etc.

Boolean Literals

Boolean literals have only two values and so are divided into two literals:

True represents a real boolean value

False represents a false boolean value

So, Boolean literals represent the logical value of either true or false. These values aren't case-sensitive and are equally valid if rendered in uppercase or lowercase mode. Boolean literals can also use the values of "0" and "1."

Examples:

```
boolean b = true;
```

```
boolean d = false;
```

TYPE CASTING IN JAVA

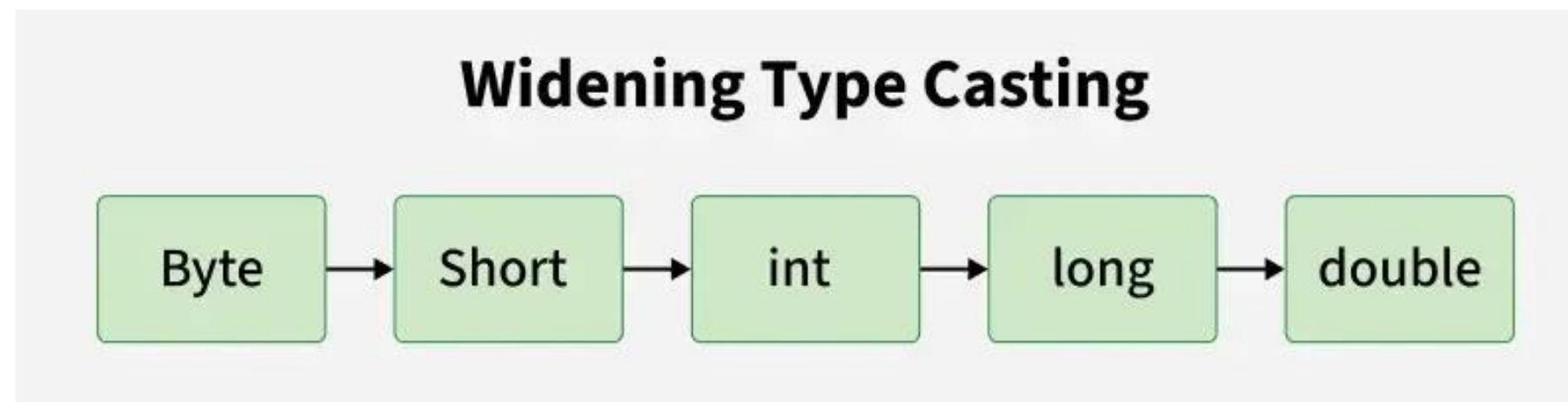
Type casting or type conversion is the fundamental concept in programming that converts one type into another. This process is crucial for ensuring compatibility and flexibility within a program.

1. Widening Type Casting (Implicit Casting/Automatic Type Conversion)

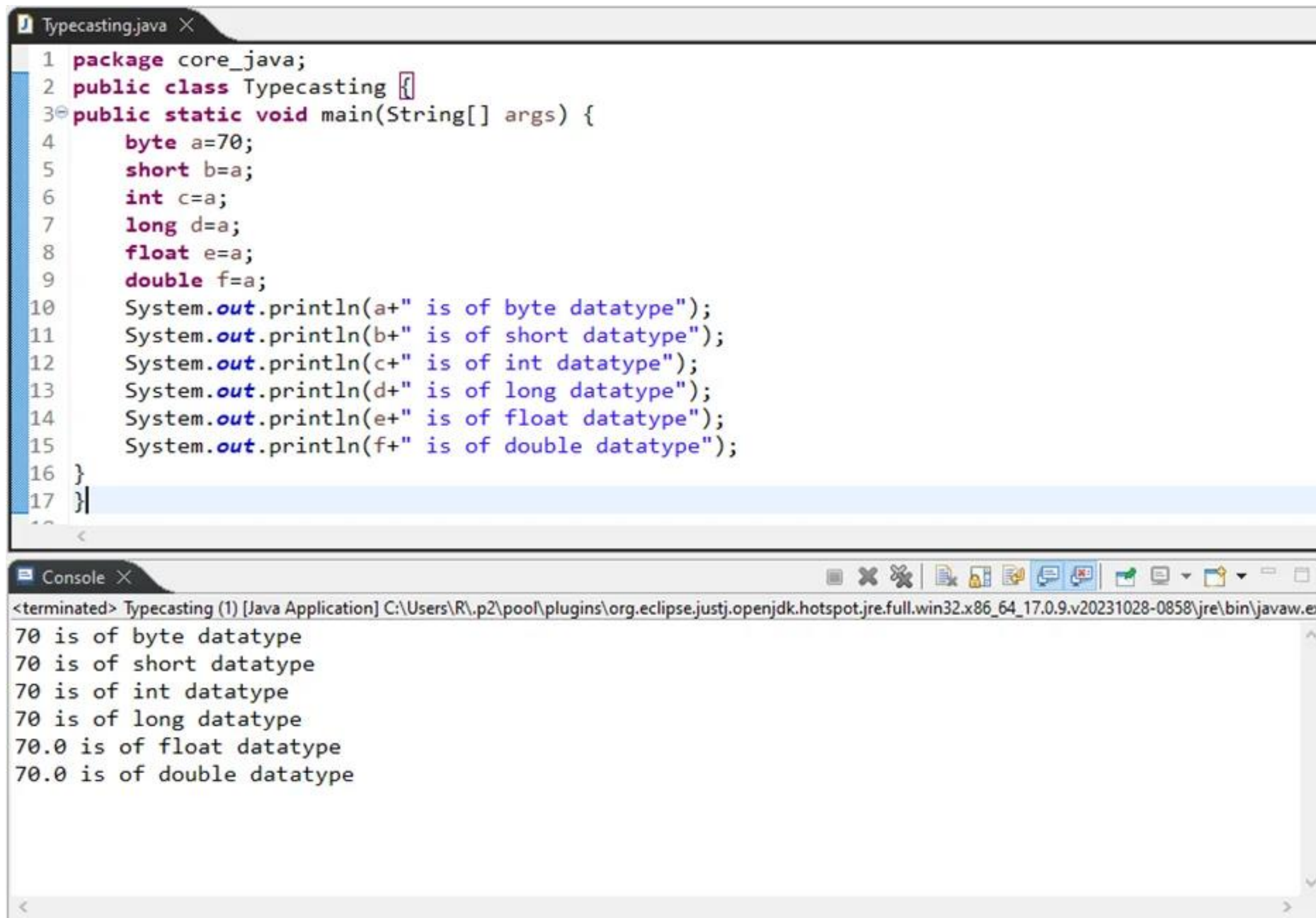
A lower data type is transformed into a higher one by a process known as widening type casting. Implicit type casting and casting down are some names for it. It occurs naturally. Since there is no chance of data loss, it is secure.

Widening Type casting occurs when:

- The target type must be larger than the source type.
- Both data types must be compatible with each other.



TYPE CASTING IN JAVA



The screenshot displays the Eclipse IDE with two panels. The top panel, titled 'Typecasting.java', contains the following Java code:

```
1 package core_java;
2 public class Typecasting {
3     public static void main(String[] args) {
4         byte a=70;
5         short b=a;
6         int c=a;
7         long d=a;
8         float e=a;
9         double f=a;
10        System.out.println(a+" is of byte datatype");
11        System.out.println(b+" is of short datatype");
12        System.out.println(c+" is of int datatype");
13        System.out.println(d+" is of long datatype");
14        System.out.println(e+" is of float datatype");
15        System.out.println(f+" is of double datatype");
16    }
17 }
```

The bottom panel, titled 'Console', shows the output of the program after execution:

```
<terminated> Typecasting (1) [Java Application] C:\Users\R\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_17.0.9.v20231028-0858\jre\bin\javaw.exe
70 is of byte datatype
70 is of short datatype
70 is of int datatype
70 is of long datatype
70.0 is of float datatype
70.0 is of double datatype
```

TYPE CASTING IN JAVA

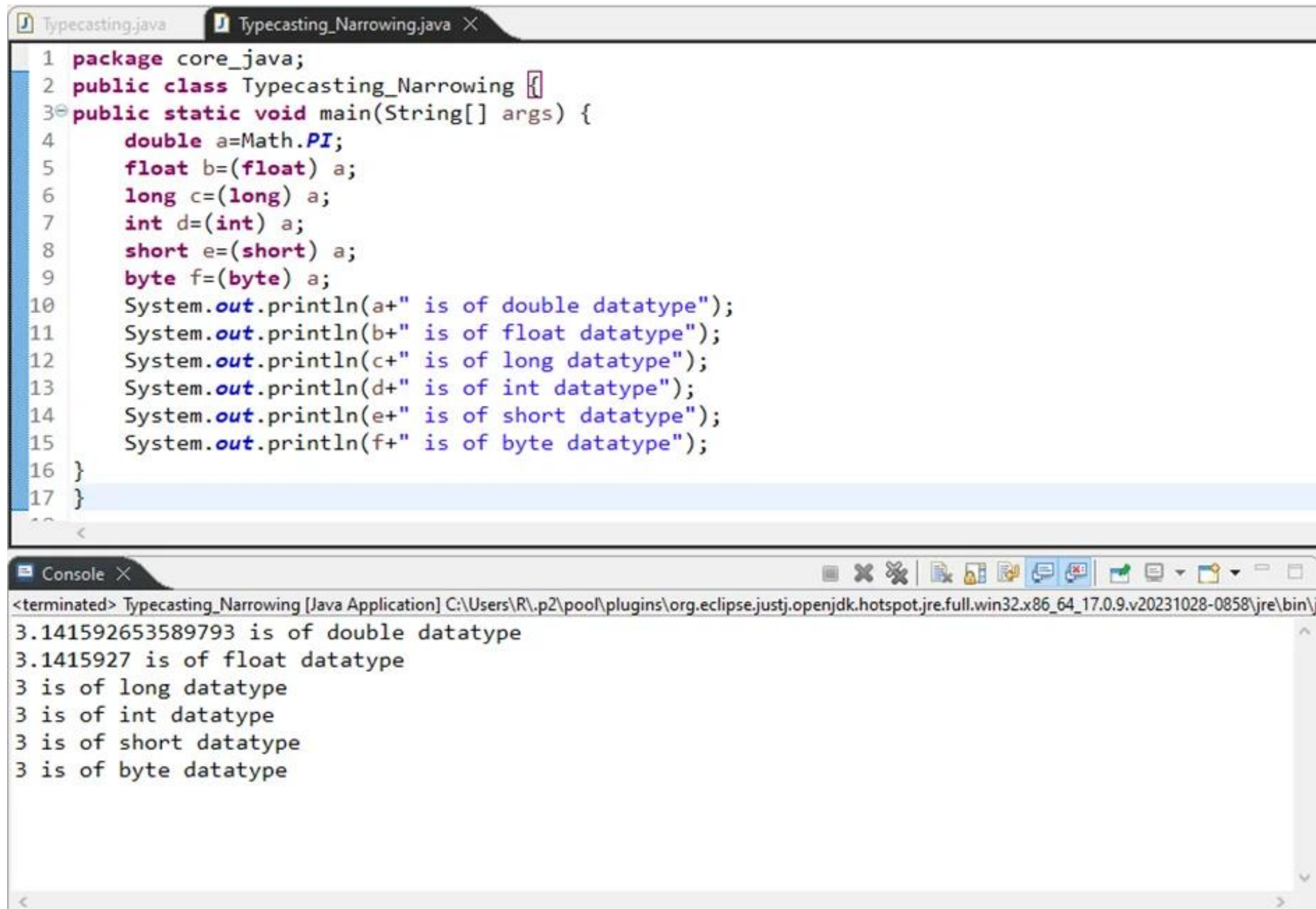
2. Narrow Type Casting (Explicit Casting)

The process of downsizing a bigger data type into a smaller one is known as narrowing type casting. Casting up or explicit type casting are other names for it. It doesn't just happen by itself. If we don't explicitly do that, a compile-time error will occur. Narrowing type casting is unsafe because data loss might happen due to the lower data type's smaller range of permitted values. A cast operator assists in the process of explicit casting.

Explicit Type Casting Order



TYPE CASTING IN JAVA



The screenshot displays the Eclipse IDE with two tabs: 'Typecasting.java' and 'Typecasting_Narrowing.java'. The 'Typecasting_Narrowing.java' tab is active, showing a Java program that demonstrates type casting. The code defines a package 'core_java', a public class 'Typecasting_Narrowing', and a main method. Inside the main method, a double variable 'a' is assigned the value of Math.PI. This double value is then cast to various primitive data types: float, long, int, short, and byte, each stored in a separate variable. The program then prints each variable along with its data type. The console output shows the results of these casts, all starting with the value 3.

```
1 package core_java;
2 public class Typecasting_Narrowing {
3     public static void main(String[] args) {
4         double a=Math.PI;
5         float b=(float) a;
6         long c=(long) a;
7         int d=(int) a;
8         short e=(short) a;
9         byte f=(byte) a;
10        System.out.println(a+" is of double datatype");
11        System.out.println(b+" is of float datatype");
12        System.out.println(c+" is of long datatype");
13        System.out.println(d+" is of int datatype");
14        System.out.println(e+" is of short datatype");
15        System.out.println(f+" is of byte datatype");
16    }
17 }
```

Console Output:

```
<terminated> Typecasting_Narrowing [Java Application] C:\Users\R\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_17.0.9.v20231028-0858\jre\bin\j
3.141592653589793 is of double datatype
3.1415927 is of float datatype
3 is of long datatype
3 is of int datatype
3 is of short datatype
3 is of byte datatype
```

USER INPUT IN JAVA

The Scanner class in Java, found in the `java.util` package, is a powerful and versatile tool for parsing primitive types and strings from various input sources. It enables your program to read and interpret user input from the console, files, or other Readable objects.

1. Import the Scanner class using `import java.util.Scanner;`
2. Create the Scanner object and connect Scanner with `System.in` by passing it as an argument i.e., `Scanner sc = new Scanner(System.in);`
3. When we want to ask the user for input, first print a prompt message so they know what to enter. Then use one of Scanner's handy methods to read the response:
 - `nextLine()`: Reads a whole line of text
 - `next()`: Reads a single word (stops at space)
 - `nextInt()`: Reads an integer
 - `nextDouble()`: Reads a decimal number
 - `nextBoolean()`: Reads a true/false value
 - `nextByte()`: Used for reading Byte value.
 - `nextFloat()`: Used for reading Float value.
 - `nextLong()`: Used for reading Long value.
 - `nextShort()`: Used for reading Short value.
4. Close the Scanner: It is good practice to close the Scanner object when you are finished using it to release system resources, by calling `scanner.close();`.

USER INPUT IN JAVA

```
package com.mcnz.example;

import java.util.Scanner; // explicit import

public class JavaScannerImportExample {

    public static void main(String args[]) {

        System.out.println("What is your favorite color?");
        Scanner stringScanner = new Scanner(System.in);
        String color = stringScanner.next();
        System.out.println(color + " is my favorite color too!");
        stringScanner.close();

    }
}
```

Java Program for practise !!!

1

Program to take marks (in double) of five subjects, cast them into int and display them.

2

Take principal, rate and time from the user and calculate simple interest.

3

Create 2 arrays for Class named StudentsData in your class – one integer array for storing roll numbers and string array for storing names. Print both arrays.

4

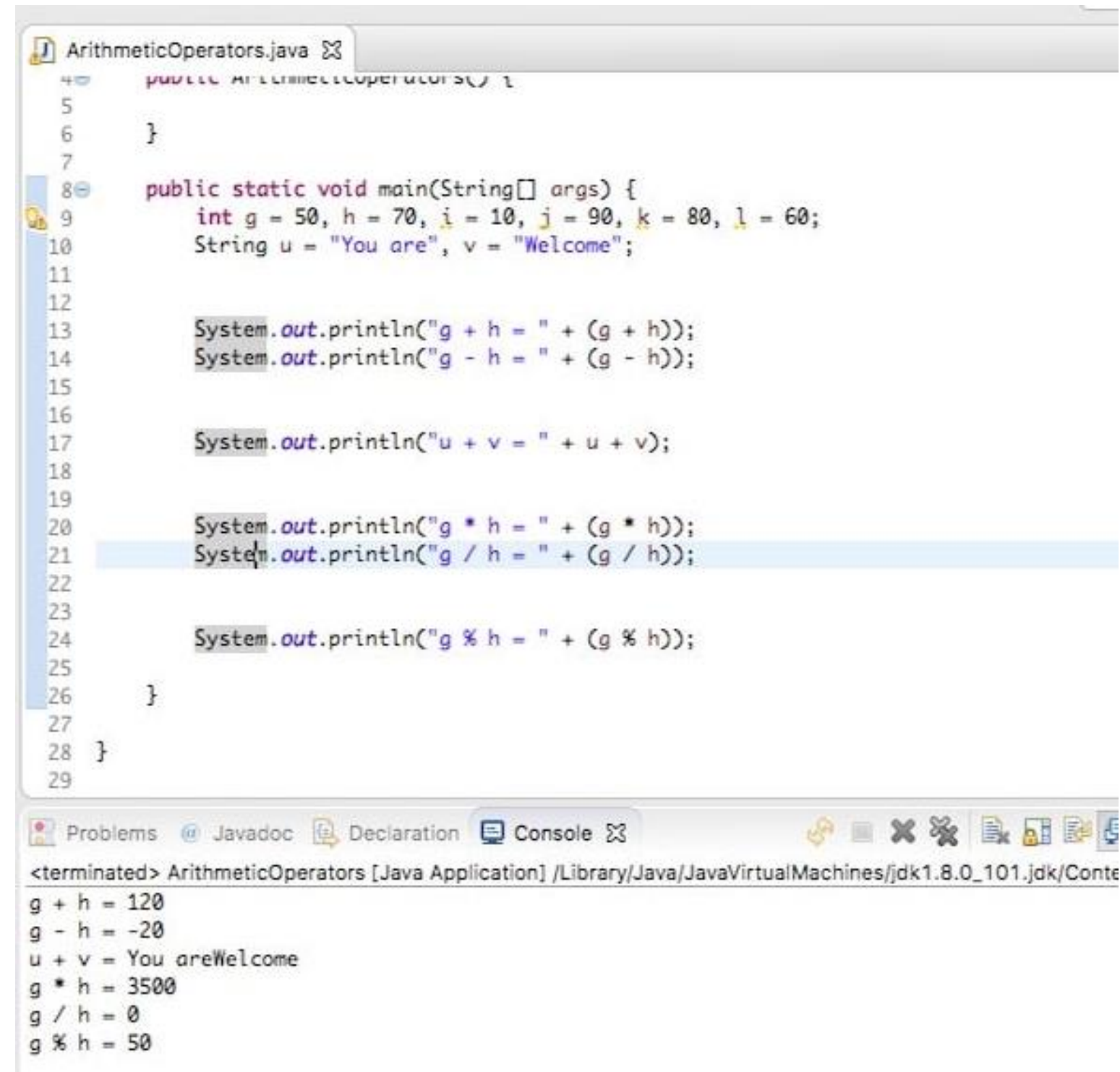
Take two integers value , divide them , show results with and without casting

Java Operators – Arithmetic Operator

1). Arithmetic Operators :-

Arithmetic operators are used to perform basic mathematical calculations. Java supports addition (+), subtraction (-), multiplication (*), division (/), and modulus (%).

- Addition (+)
- Subtraction (-)
- Multiplication (*)
- Division (/)
- Modulus (%): Returns the remainder of a division operation



```
ArithmeticOperators.java
4 public ArithmeticOperators() {
5
6 }
7
8 public static void main(String[] args) {
9     int g = 50, h = 70, i = 10, j = 90, k = 80, l = 60;
10    String u = "You are", v = "Welcome";
11
12
13    System.out.println("g + h = " + (g + h));
14    System.out.println("g - h = " + (g - h));
15
16
17    System.out.println("u + v = " + u + v);
18
19
20    System.out.println("g * h = " + (g * h));
21    System.out.println("g / h = " + (g / h));
22
23
24    System.out.println("g % h = " + (g % h));
25
26 }
27
28 }
29
```

Problems Javadoc Declaration Console

<terminated> ArithmeticOperators [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_101.jdk/Conte

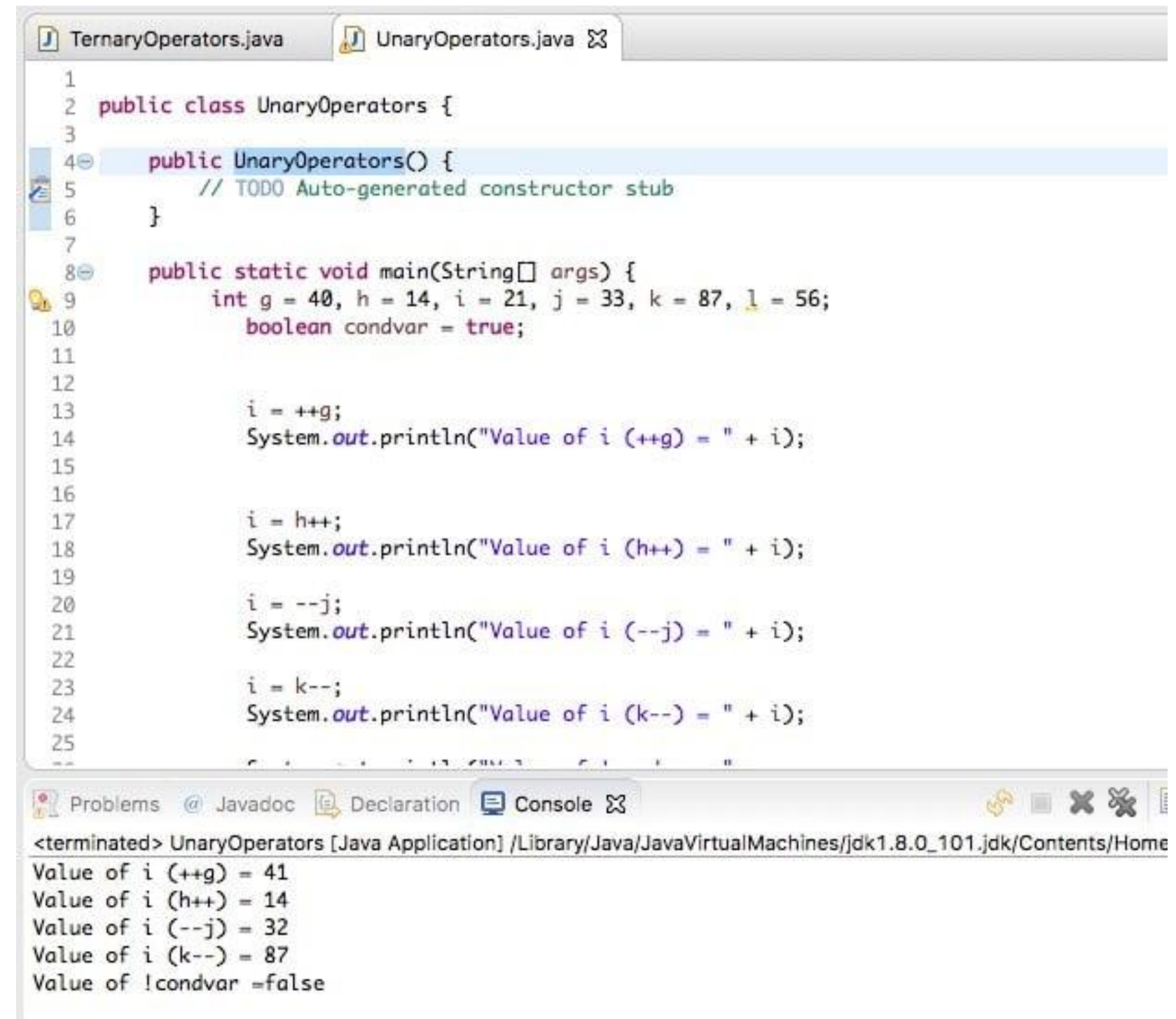
```
g + h = 120
g - h = -20
u + v = You areWelcome
g * h = 3500
g / h = 0
g % h = 50
```

Java Operators – Unary Operator

2. Unary Operators :-

Unary Operators need only one operand. They are used to increment, decrement, or negate a value.

- ++ , Increments by 1.
 - ☐ Post-Increment: Uses value first, then increments.
 - ☐ Pre-Increment: Increments first, then uses value.
- -- , Decrements by 1.
 - ☐ Post-Decrement: Uses value first, then decrements.
 - ☐ Pre-Decrement: Decrements first, then uses value.



```
1 public class UnaryOperators {
2
3
4 public UnaryOperators() {
5     // TODO Auto-generated constructor stub
6 }
7
8 public static void main(String[] args) {
9     int g = 40, h = 14, i = 21, j = 33, k = 87, l = 56;
10    boolean condvar = true;
11
12
13    i = ++g;
14    System.out.println("Value of i (++g) = " + i);
15
16
17    i = h++;
18    System.out.println("Value of i (h++) = " + i);
19
20    i = --j;
21    System.out.println("Value of i (--j) = " + i);
22
23    i = k--;
24    System.out.println("Value of i (k--) = " + i);
25
26}
```

Problems @ Javadoc Declaration Console

<terminated> UnaryOperators [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_101.jdk/Contents/Home

Value of i (++g) = 41
Value of i (h++) = 14
Value of i (--j) = 32
Value of i (k--) = 87
Value of !condvar =false

Java Operators – Assignment Operator

3). Java Assignment Operators :-

'=' The assignment operator is used to assign a value to any variable. It has right-to-left associativity, i.e. value given on the right-hand side of the operator is assigned to the variable on the left, and therefore right-hand side value must be declared before using it or should be a constant.

Simple assignment operator is:

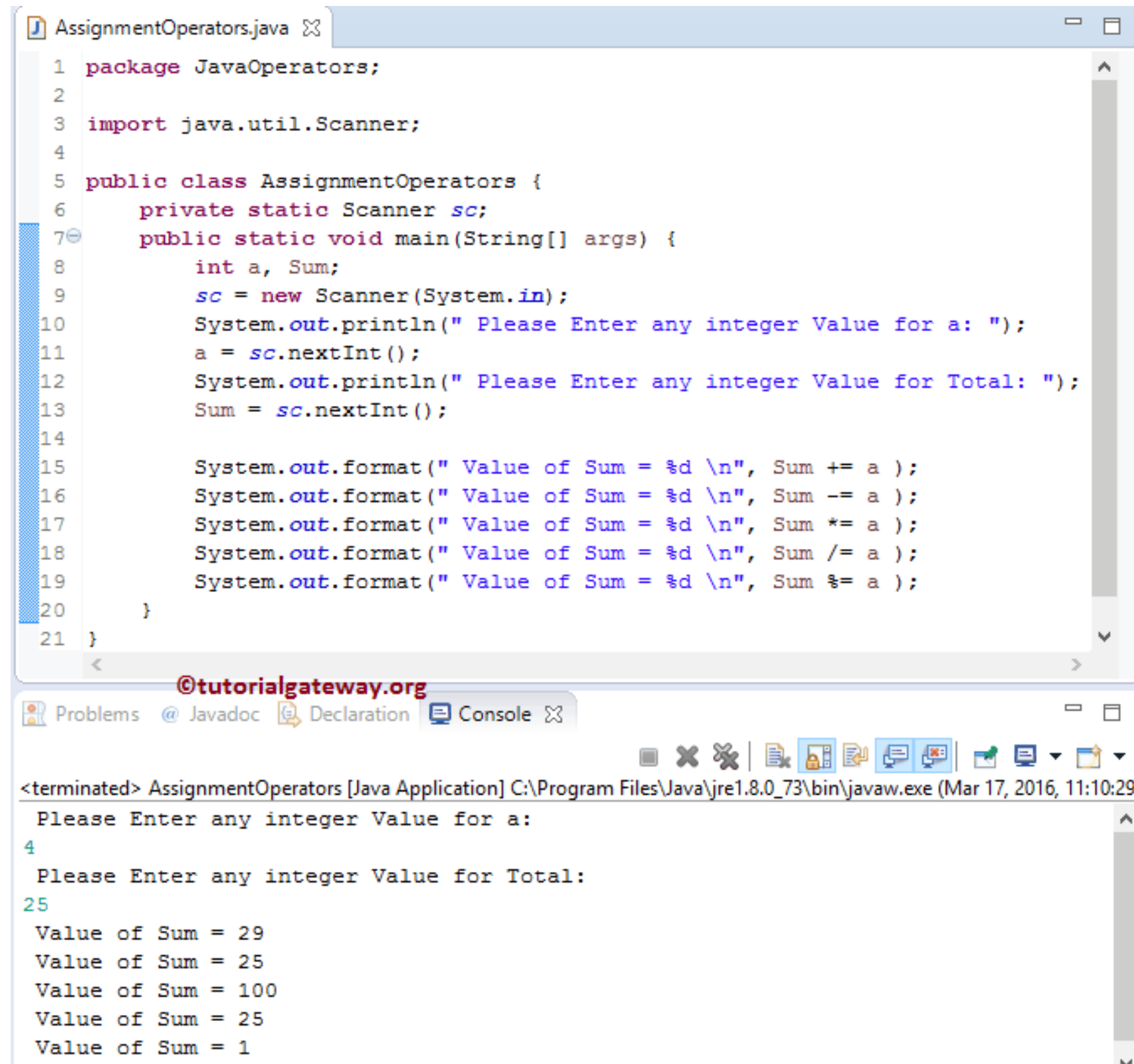
variable = value;

In many cases, the assignment operator can be combined with others to create shorthand compound statements. For example, `a += 5` replaces `a = a + 5`.

Common compound operators include:

- `+=` , Add and assign.
- `-=` , Subtract and assign.
- `*=` , Multiply and assign.
- `/=` , Divide and assign.
- `%=` , Modulo and assign.

Java Operators – Assignment Operator Eg



The screenshot displays an IDE window titled "AssignmentOperators.java". The code defines a class `AssignmentOperators` with a `main` method. It uses a `Scanner` to take two integer inputs, `a` and `Sum`. The program then performs five assignment operations: `Sum += a`, `Sum -= a`, `Sum *= a`, `Sum /= a`, and `Sum %= a`, printing the result of each operation. The console output shows the results of these operations for inputs 4 and 25.

```
1 package JavaOperators;
2
3 import java.util.Scanner;
4
5 public class AssignmentOperators {
6     private static Scanner sc;
7     public static void main(String[] args) {
8         int a, Sum;
9         sc = new Scanner(System.in);
10        System.out.println(" Please Enter any integer Value for a: ");
11        a = sc.nextInt();
12        System.out.println(" Please Enter any integer Value for Total: ");
13        Sum = sc.nextInt();
14
15        System.out.format(" Value of Sum = %d \n", Sum += a );
16        System.out.format(" Value of Sum = %d \n", Sum -= a );
17        System.out.format(" Value of Sum = %d \n", Sum *= a );
18        System.out.format(" Value of Sum = %d \n", Sum /= a );
19        System.out.format(" Value of Sum = %d \n", Sum %= a );
20    }
21 }
```

©tutorialgateway.org

Problems @ Javadoc Declaration Console

<terminated> AssignmentOperators [Java Application] C:\Program Files\Java\jre1.8.0_73\bin\javaw.exe (Mar 17, 2016, 11:10:29)

```
Please Enter any integer Value for a:
4
Please Enter any integer Value for Total:
25
Value of Sum = 29
Value of Sum = 25
Value of Sum = 100
Value of Sum = 25
Value of Sum = 1
```


Java Operators – Assignment Operator Eg

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
&=	x &= 3	x = x & 3
=	x = 3	x = x 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3

Java Operators – Relational Operator

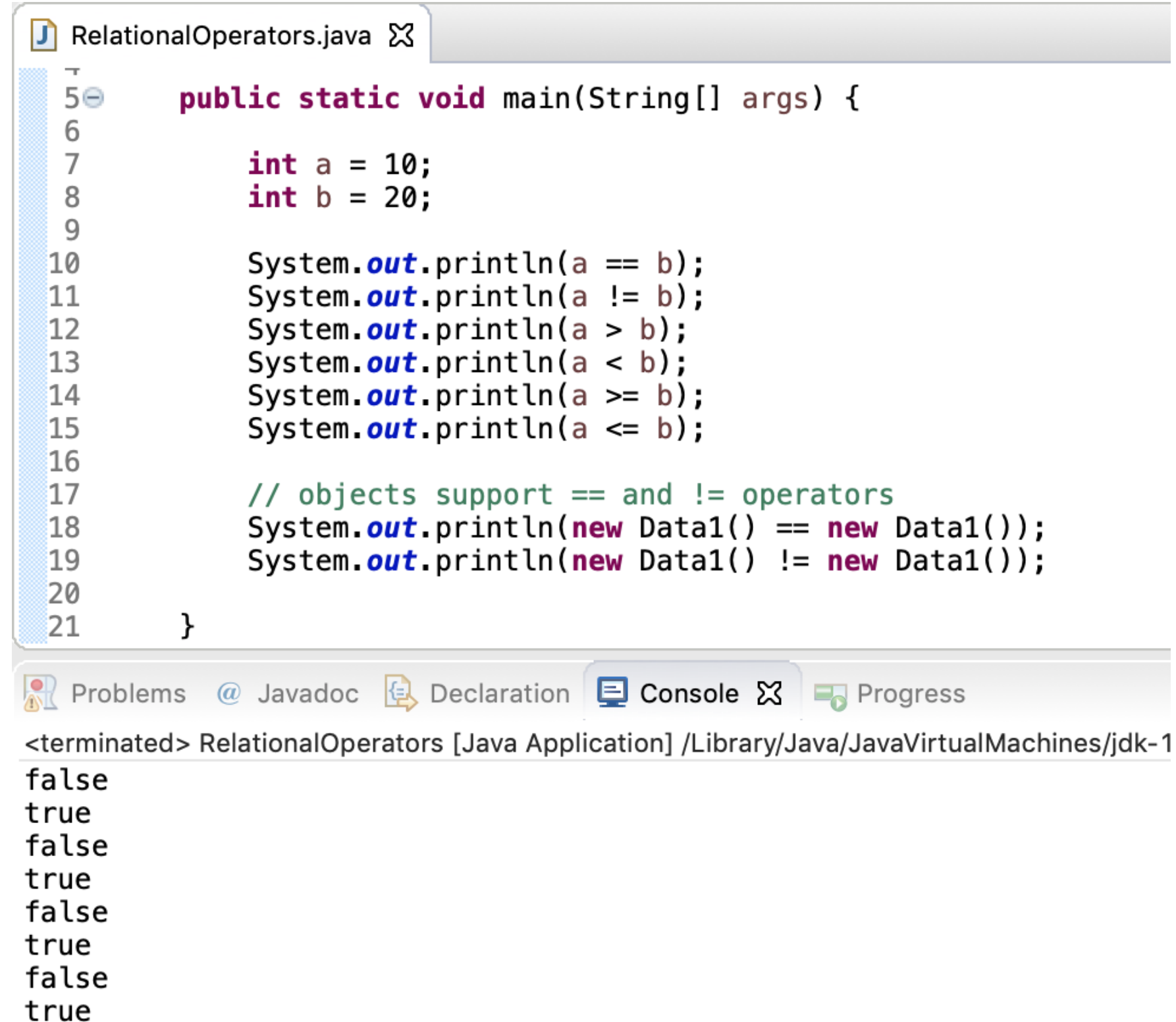
4). Relational Operators/Comparison operators :-

Relational Operators are used to check for relations like equality, greater than and less than. They return boolean results after the comparison and are extensively used in looping statements as well as conditional if-else statements. This is important in programming, because it helps us to find answers and make decisions.

The return value of a comparison is either true or false. These values are known as Boolean. Relational operators compare values and return Boolean results.

- == , Equal to.
- != , Not equal to.
- < , Less than.
- <= , Less than or equal to.
- > , Greater than.
- >= , Greater than or equal to.

Java Operators – Relational Operator Eg



```
RelationalOperators.java ✕  
5 public static void main(String[] args) {  
6  
7     int a = 10;  
8     int b = 20;  
9  
10    System.out.println(a == b);  
11    System.out.println(a != b);  
12    System.out.println(a > b);  
13    System.out.println(a < b);  
14    System.out.println(a >= b);  
15    System.out.println(a <= b);  
16  
17    // objects support == and != operators  
18    System.out.println(new Data1() == new Data1());  
19    System.out.println(new Data1() != new Data1());  
20  
21 }
```

Problems Javadoc Declaration Console ✕ Progress

<terminated> RelationalOperators [Java Application] /Library/Java/JavaVirtualMachines/jdk-1

false
true
false
true
false
true
false
true

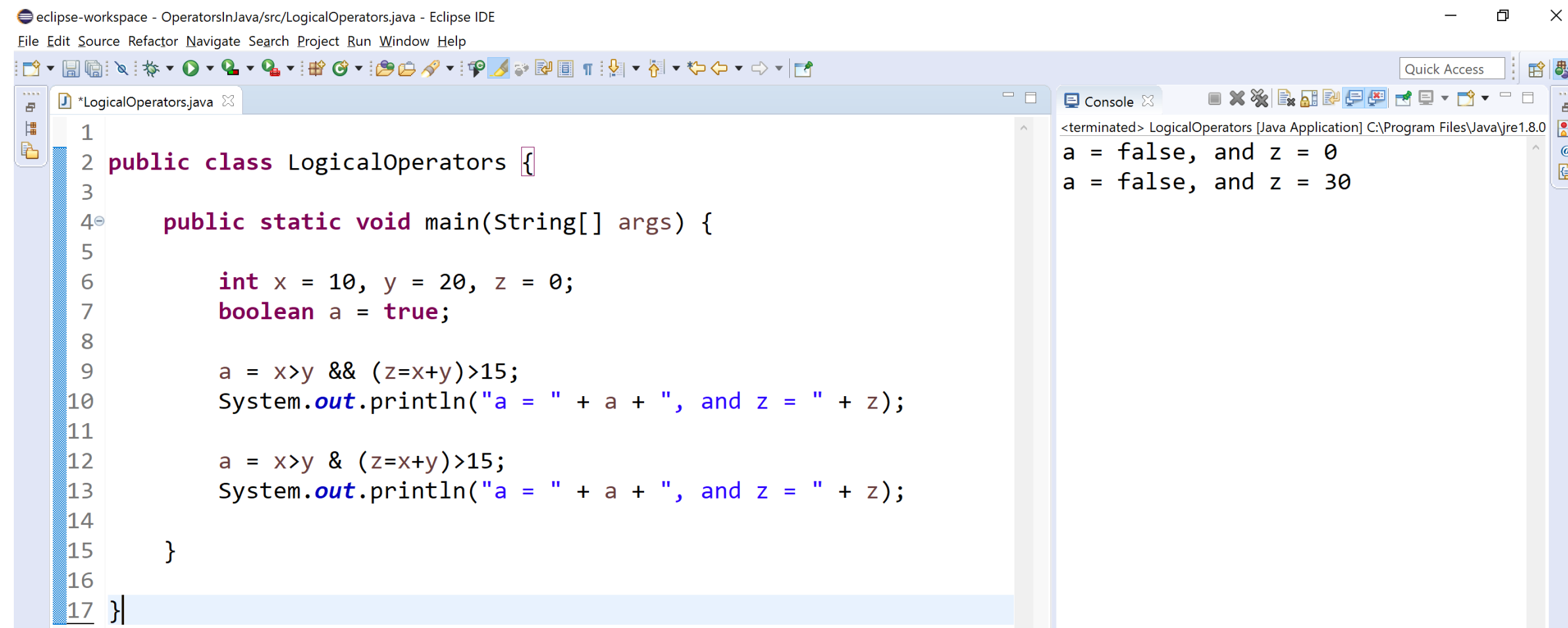
Java Operators – Logical Operator

5. Logical Operators :-

Logical Operators are used to perform "logical AND" and "logical OR" operations, similar to AND gate and OR gate in digital electronics. They have a short-circuiting effect, meaning the second condition is not evaluated if the first is false.

Conditional operators are:

- && Logical AND returns true when both conditions are true.
- || Logical OR returns true if at least one condition is true.
- ! Logical NOT returns true when a condition is false and vice-versa



```
eclipse-workspace - OperatorsInJava/src/LogicalOperators.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help

*LogicalOperators.java
1
2 public class LogicalOperators {
3
4     public static void main(String[] args) {
5
6         int x = 10, y = 20, z = 0;
7         boolean a = true;
8
9         a = x>y && (z=x+y)>15;
10        System.out.println("a = " + a + ", and z = " + z);
11
12        a = x>y & (z=x+y)>15;
13        System.out.println("a = " + a + ", and z = " + z);
14
15    }
16
17 }
```

```
<terminated> LogicalOperators [Java Application] C:\Program Files\Java\jre1.8.0
a = false, and z = 0
a = false, and z = 30
```

Java Operators – Ternary Operator

6). Ternary Operator :-

A ternary operator evaluates the test condition and executes a block of code based on the result of the condition. Its syntax is : condition ? expression1 : expression2;

The condition is evaluated and

- if condition is true, expression1 is executed.
- And, if condition is false, expression2 is executed.

The ternary operator takes 3 operands (condition, expression1, and expression2). Hence, the name ternary operator.

```
1 import java.util.Scanner;
2 public class Example {
3     private static Scanner sc;
4
5     public static void main(String[] args) {
6         int Num;
7         sc = new Scanner(System.in);
8         System.out.print("Please Enter the any integer Value: ");
9         Num = sc.nextInt();
10
11         System.out.println((Num >=0)? "\nPOSITIVE":"\nNEGATIVE");
12     }
13 }
```

Problems Console X

<terminated> Example [Java Application] /Users/sureshbabu/.p2/pool/plugins/org.eclipse.justj.openjdk.hotspot.jre

Please Enter the any integer Value: 215

POSITIVE

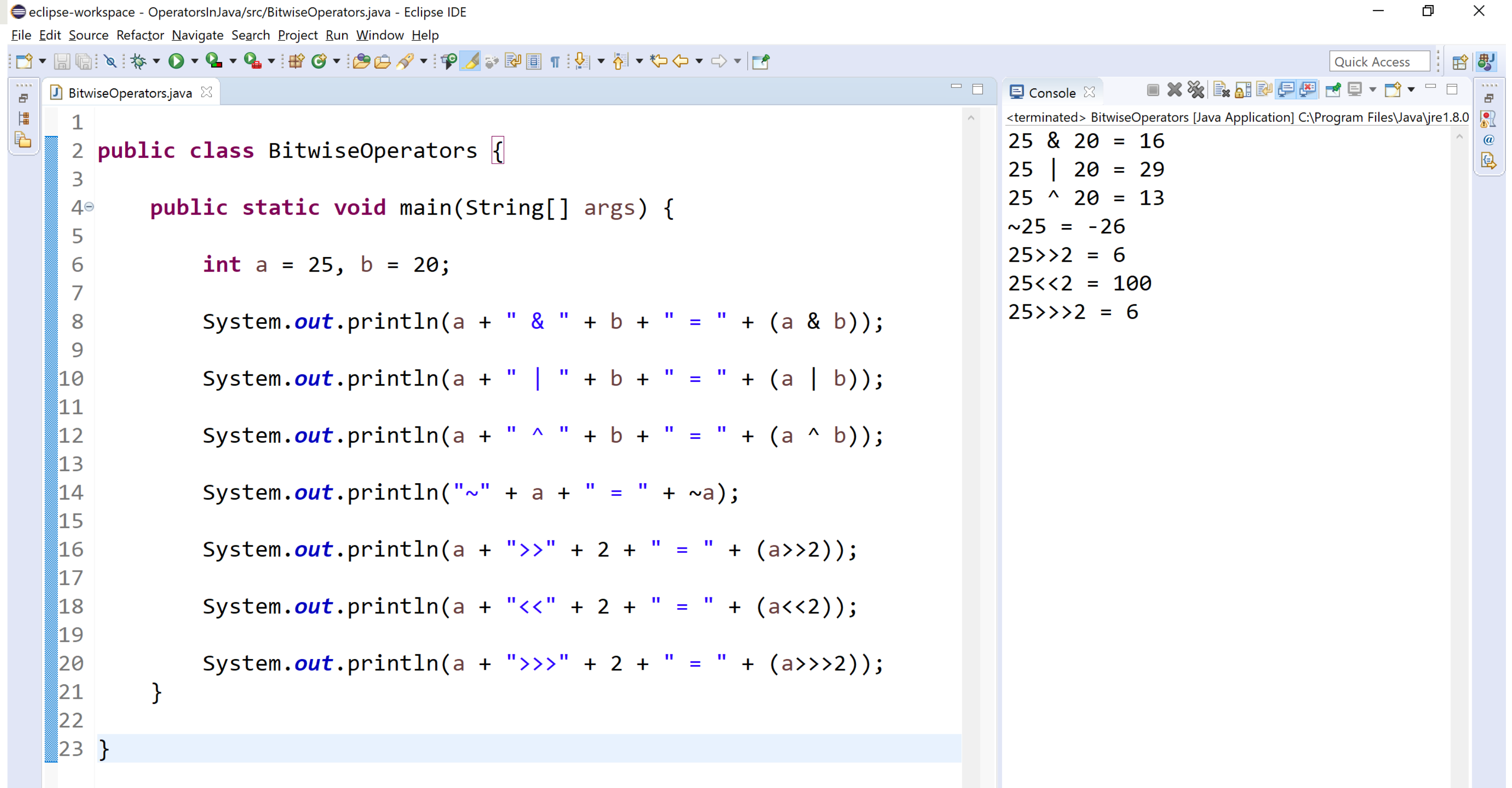
Java Operator – Bitwise Operators

7). Bitwise Operators :-

Bitwise Operators are used to perform the manipulation of individual bits of a number and with any of the integer types.

- Bitwise AND (&): This operator takes two numbers and performs a logical AND operation on each corresponding bit. The resulting bit is 1 only if both bits are 1; otherwise, it's 0.
- Bitwise OR (|): This operator takes two numbers and performs a logical OR operation on each corresponding bit. The resulting bit is 1 if at least one of the bits is 1; otherwise, it's 0.
- Bitwise XOR (^): This operator takes two numbers and performs a logical XOR (exclusive OR) operation on each corresponding bit. The resulting bit is 1 if two bits are different; otherwise, it's 0.
- Bitwise NOT (~): This is a unary operator that takes one number and inverts all of its bits (flips 0s to 1s and 1s to 0s). The result is typically interpreted in two's complement form, leading to a negative value for positive inputs.
- Left Shift (<<): This operator shifts the bits of the first operand to the left by the number of positions specified by the second operand. The empty positions on the right are filled with zeros. This effectively multiplies the number by powers of 2.
- Signed Right Shift (>>): This operator shifts the bits of first operand to right by the number of positions specified by second operand. The empty positions on the left are filled with sign bit (0 for positive numbers, 1 for negative numbers). This effectively divides number by powers of 2 while preserving the sign.
- Unsigned Right Shift (>>>): This operator shifts bits of first operand to right by number of positions specified by second operand. The empty positions on left are always filled with zeros, regardless of number's sign. This is useful for manipulating bit patterns without regard to signedness.

Java fundamental rules and conventions



The screenshot shows the Eclipse IDE with a Java file named `BitwiseOperators.java` open. The code defines a `public class BitwiseOperators` with a `main` method. Inside `main`, two integers `a = 25` and `b = 20` are declared. The program then prints the results of several bitwise operations: `a & b`, `a | b`, `a ^ b`, `~a`, `a >> 2`, `a << 2`, and `a >>> 2`. The console on the right shows the output of these operations.

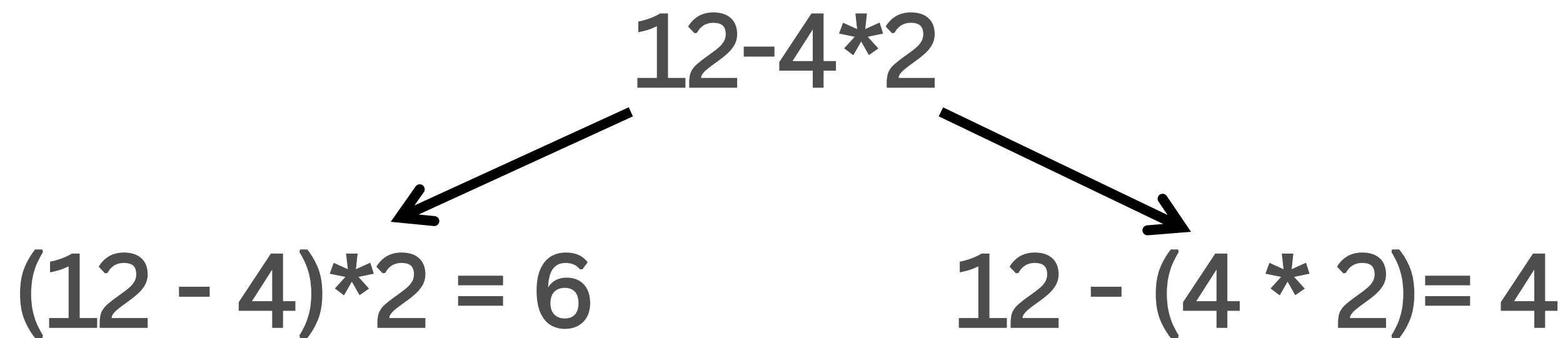
```
1
2 public class BitwiseOperators {
3
4     public static void main(String[] args) {
5
6         int a = 25, b = 20;
7
8         System.out.println(a + " & " + b + " = " + (a & b));
9
10        System.out.println(a + " | " + b + " = " + (a | b));
11
12        System.out.println(a + " ^ " + b + " = " + (a ^ b));
13
14        System.out.println("~" + a + " = " + ~a);
15
16        System.out.println(a + ">>" + 2 + " = " + (a>>2));
17
18        System.out.println(a + "<<" + 2 + " = " + (a<<2));
19
20        System.out.println(a + ">>>" + 2 + " = " + (a>>>2));
21    }
22
23 }
```

Console Output:

```
<terminated> BitwiseOperators [Java Application] C:\Program Files\Java\jre1.8.0
25 & 20 = 16
25 | 20 = 29
25 ^ 20 = 13
~25 = -26
25>>2 = 6
25<<2 = 100
25>>>2 = 6
```

Operator Precedence & Associativity in Java

- **Precedence and associative rules** are applied when dealing with hybrid equations that involve more than one type of operator. In such cases, these rules determine which part of the equation to consider first, as there can be many different valuations for the same equation.
- **Operator Precedence** : Operator Precedence specifies the order in which operations are performed within an expression. When an expression contains multiple operators, those with higher precedence are evaluated before those with lower precedence.
- The table below illustrates the precedence of operators in decreasing order of magnitude, with the top row representing the highest precedence and the bottom row showing the lowest precedence.



Operator Precedence Table

Operators	Associativity	Type
++ --	Right to left	Unary postfix
++ -- + - ~ ! (type)	Right to left	Unary prefix
* / %	Left to right	Multiplicative
+ -	Left to right	Additive
<< >> >>>	Left to right	Shift
< <= > >=	Left to right	Relational
== !=	Left to right	Equality
&	Left to right	Boolean Logical AND
^	Left to right	Boolean Logical Exclusive OR
	Left to right	Boolean Logical Inclusive OR
&&	Left to right	Conditional AND
	Left to right	Conditional OR
?:	Right to left	Conditional
= += -= *= /= %=	Right to left	Assignment

Operator Associativity in Java

Operator associativity in Java determines the order of evaluation when multiple operators of the same precedence appear in an expression.

Java supports two types of associativity:

1. Left-to-Right Associativity

When multiple operators of the same precedence appear in an expression, they are evaluated from left to right. For example, in the expression $a + b - c$, addition and subtraction have the same precedence and are left-associative, so the expression is evaluated as $(a + b) - c$.

Operators with left-to-right associativity include:

- Arithmetic operators: $+$, $-$, $*$, $/$, $\%$
- Relational operators: $>$, $<$, $>=$, $<=$
- Logical AND/OR: $\&\&$, $\|\|$
- Bitwise operators: $\&$, $\|$, $\wedge$, $\ll$, $\gg$

2. Right-to-Left Associativity

Right-to-Left Associativity means that operators are evaluated from right to left when they have the same precedence.

Operators with right-to-left associativity include:

- Assignment operators: $=$, $+=$, $-=$, etc.
- Unary operators: $++$, $--$, $!$, $\sim$

Conditional Statements in Java

1. if statement: Executes a block of code if a condition is true.

```
if (condition) {  
    // code to be executed if condition is true  
}
```

The condition inside the if statement must result in a boolean value - it can be either a boolean expression (like $x > y$) or a boolean variable (like `isLightOn`).

Also if is in lowercase letters. Uppercase letters (If or IF) will generate an error.

2. if-else statement: Executes one block of code if a condition is true and another block if the condition is false.

```
if (condition) {  
    // code to be executed if condition is true  
} else {  
    // code to be executed if condition is false  
}
```


Conditional Statements in Java

3. if-else-if ladder: Used to test multiple conditions sequentially. Use the else if statement to specify a new condition if the first condition is false.

```
if (condition1) {  
    // code for condition1  
} else if (condition2) {  
    // code for condition2  
} else {  
    // code if no conditions are met  
}
```

How it works:

- Top-down evaluation : The conditions within the if-else-if ladder are evaluated from top to bottom.
- First true condition : As soon as an if or else if condition evaluates to true, the corresponding block of code is executed.
- Bypass the rest : Once a true condition is found and its block is executed, the rest of the if-else-if ladder is bypassed, and the program continues execution after the ladder.
- Optional else block : If none of the if or else if conditions are true, an optional final else block can be included. This else block acts as a default and will be executed only if all preceding conditions are false.

Conditional Statements in Java

4. switch statement: Provides way to select one of many possible execution paths based on value of a variable.

```
switch (variable) {  
    case value1:  
        // code for value1  
        break;  
    case value2:  
        // code for value2  
        break;  
    default:  
        // code if no case matches  
}
```

How it works:

- The switch expression is evaluated once.
- The result is compared with each case value.
- If there is a match, the matching block of code runs.
- The break statement stops the switch after the matching case has run.
- The default statement runs if there is no match.

Looping Statements (Iterative Statements)

These statements repeatedly execute a block of code as long as a certain condition remains true.

1. **for loop** : Used when the number of iterations is known.

```
for (initialization; condition; increment/decrement) {  
    // code to be executed repeatedly  
}
```
2. **while loop** : Executes a block of code as long as a condition is true.

```
while (condition) {  
    // code to be executed repeatedly  
}
```
3. **do-while loop**: Similar to while, but guarantees at least one execution of the loop body before checking the condition.

```
do {  
    // code to be executed repeatedly  
} while (condition);
```

Jump Statements in Java - break

Jumping statements are control statements that transfer execution control from one point to another point in program. There are three Jump statements that are provided in Java programming language :

1. **Break statement**: In java, the break statement is used to terminate the execution of the nearest looping statement or switch statement.
 - The break statement is used to terminate loops - for loop, while loop and do-while-loop, the control reaches the statement that follows the loop.
 - SWITCH - The break statement terminates the switch case after execution of that block , preventing fall –through. In switch statement, if we don't use break, once a matching case is found, the execution continues into the next cases until a break or the end of the switch is reached, this behavior is called fall- through.
 - SYNTAX -
 break;
 - "break" is a reserved keyword means we can't use it as an identifier.

Jump Statements in Java - Break

Break statement Example(in Loop):

```
Class LoopBreak{
    public static void main(String[] args){
        int n=10;
        for(int i=0; i<n; i++){
            if (i==6)
                break;
            System.out.println(i);
        }
    }
}
```

Break statement Example(in Switch):

```
Class SwitchBreak{
    public static void main(String[] args){
        int day =3;
        switch(day){
            case 1 :
                System.out.println("Monday");
                break;
            case 2 :
                System.out.println("Tuesday");
                break;
            case 3 :
                System.out.println("Wednesday");
                break;
            default:
                System.out.println("Other day");
        }
    }
}
```

Jump Statements in Java - Continue

2. Continue statement :

- The continue statement is used within loops to skip the remaining statements of the current iteration and process the next iteration of the loop.
- The continue statement does not terminate the loop. Instead, it skips the current iteration and moves to the next iteration of the loop.
- "continue" is a reserved keyword means we can't use it as an identifier.

Continue Statement Example:

```
Class LoopContinue{  
    public static void main(String[] args){  
        int n=10;  
        for(int i=0; i<n; i++){  
            if (i==6)  
                Continue;  
            System.out.println(i);  
        }  
    }  
}
```


Jump Statements in Java - Return

3. Return Statement :

- The "return" keyword can help you transfer control from one method to the method that called it. Since the control jumps from one part of the program to another, the return is also a jump statement.
- "return" is a reserved keyword means we can't use it as an identifier.
- It is used to exit from a method, with or without a value.
- The return keyword exists the entire method immediately.

Return Statement Example:

```
Class LoopContinue{
    public static void main(String[] args){
        int n=10;
        for(int i=0; i<n; i++){
            if (i==6)
                return;
            System.out.println(i);
        }
        System.out.println("This is inside main
        method");
    }
    System.out.println("This is outside main method");
}
```


Recursion in Java

Recursion in Java is a programming technique where a method calls itself to solve a problem. It is particularly useful for problems that can be broken down into smaller, self-similar subproblems.

Key Components of Recursion :

1. Base Case : Every recursive method must have a base case, which is a condition that stops the recursion. This condition defines the simplest instance of the problem that can be solved directly without further recursive calls. Without a base case, the recursion would continue indefinitely, leading to a `StackOverflowError`.
2. Recursive Case : The recursive case is where the method calls itself with a modified input that moves closer to the base case. This call breaks down the larger problem into a smaller, similar subproblem.

Advantages of Recursion :

1. Readability : For certain problems, recursive solutions can be more concise and easier to understand than iterative ones.
2. Natural Fit for Tree/Graph Traversal : Recursion aligns well with hierarchical nature of trees and graphs.

Disadvantages of Recursion :

1. Memory Consumption : Each recursive call adds a new stack frame, potentially leading to high memory usage and `StackOverflowError` for deep recursions.
2. Performance Overhead : The overhead of function calls can make recursive solutions less efficient than iterative ones in some cases.

Recursion in Java

- **CODE (Factorial of number n)**

```
Static int factorial (int n){  
    If (n == 0 || n ==1){           //base case  
        return 1;  
    }  
    return n * factorial (n-1); //recursive case  
}
```
- For n = 5 ,
 Factorial (5) = 5 * factorial (4)
 Factorial (4) = 5 * factorial (3)
 Factorial (3) = 5 * factorial (2)
 Factorial (2) = 5 * factorial (1)
 Factorial (1) = Base Case Reached = 1
- As calls are made methods are stacked,
 they are pushed (added) to stack.

factorial (1) - Top

factorial (2)

factorial (3)

factorial (4)

factorial (5) - Bottom

Factorial(1) = 1

Factorial(2) = 2 * 1 = 2

Factorial(3) = 3 * 2 = 6

Factorial(4) = 4 * 6 = 24

Factorial(5) = 5 * 24 = 120

Iteration in Java

Iterations refer to the process of repeatedly executing a block of code for each element in a sequence or collection. This can be achieved through various looping constructs, such as:

1. for loop : Used when the number of iterations is known or can be easily determined.
2. while loop : Used when the number of iterations is not fixed but the loop should continue as long as a condition is true.
3. Do while loop : Used when the loop body should execute atleast once, even if condition is false.

Advantages of Iterations :

1. Iteration can be used to repeatedly execute a set of statements without the overhead of function calls and without using stack memory.
2. Iteration is faster and more efficient than recursion.
3. It's easier to optimize iterative codes, and they generally have polynomial time complexity.
4. They are used to iterate over the elements present in data structures like an array, set, map, etc.

CODE (Factorial of number n)

```
Static int factorial (int n){  
    int result = 1 ;  
    int i;  
    for(i=1; i<=n; i++){  
        result = result*i; }  
    return result;}
```

Recursion V/S Iterations

When to Use Recursion?

- Sometimes we encounter a problem that is too complex to solve directly. In most cases, we try to break such problems down into smaller pieces. Then, we can find a way to solve these smaller pieces and then build up a solution to the entire problem. This is the entire idea behind recursion.
- Recursion is when a function calls itself directly or indirectly, and the function calling itself is called a recursive function. It is mainly used when the solution to a bigger problem can be expressed in terms of smaller problems.
- To terminate the recursion, we use some conditions so that the function knows when not to make any further recursive call to itself and return; otherwise, it will lead to infinite recursion once the function is called. Such conditions are called base conditions in recursion.

When to Use Iteration?

- Iteration is when we execute a set of instructions repeatedly until the condition controlling the loop becomes false. It includes initialization, comparison, executing statements inside the iteration, and updating the control variable.
- In iteration, it is necessary to have the right controlling condition. Otherwise, the program may go into an infinite loop.

Recursion V/S Iteration in Java

RECURSION	ITERATION
A method that calls itself until base condition is reached.	Repeated execution of a block using loops(for, while, do while).
Achieved through method calls.	Achieved through loop constructs.
Recursion program has base condition & recursion case.	Iteration has loop condition, else causes infinite loops.
Memory usage is more in recursion because each call creates new stack frame until base case is reached.	Iteration reuses the same memory space with loop variables, so its memory usage stays constant and is more efficient.
Both Time Complexity is Higher or same. Space Complexity is generally either higher, can be same in same cases where we need an explicit stack	Both Time Complexity is Lower or same. Space Complexity is generally lower.
Preferred when problem is naturally recursive	Proffered when performance and memory efficiency are required

Arrays in Java

Arrays in Java are fundamental data structures used to store multiple values of the same data type in a single variable. They are fixed-size sequential collections of elements, meaning their length is established upon creation and cannot be changed afterward.

One - Dimensional Arrays:

1. Declaring Arrays :

```
type var-name[ ];
```

Eg :- `int month_days[];`

This declaration establishes the fact that `month_days` is an array variable, no array actually exists.

2. Instantiating Arrays :

```
int arr[] = new int[size];
```

Eg :- `month_days = new int[12];`

Use `new` to allocate an array, we must specify the type and number of elements to allocate.

3. Initializing Arrays :

```
int month_days[] = { 31, 28, 31, 30, 31, 30, 31, 31, 30, 31,30, 31 };
```

An array initializer is a list of comma-separated expressions surrounded by curly braces.

4. Accessing Array Elements :

```
month_days[3];
```


Arrays in Java

5. **Modifying Array Elements** : Modify any element in an array by directly accessing it using its index.

```
month_days[3] = 28;
```

5. **Iterating through Arrays** : Iterate an array using either Loops or Arrays.toString

6. **Array Length** : Each array has a property called .length, which tells us the number of elements .

```
month_days.length;
```

Multi - dimensional Arrays

In Java, multidimensional arrays are actually arrays of arrays. To declare a multidimensional array variable, specify each additional index using another set of square brackets. Java also supports multi-dimensional arrays, which are essentially arrays of arrays, like 2D arrays (matrices) or 3D arrays.

1. **Declaring Arrays** :

```
int twoD[][];
```

2. **Instantiating Arrays** :

```
int twoD[][] = new int[4][5];
```

3. **Initializing Arrays** :

```
int twoD[][] = {  
    { 0, 1, 2, 3, 4 }, { 5, 6, 7, 8, 9 }, {10, 11, 12, 13, 14}, {15, 16, 17, 18, 19}  
};
```